VIETNAM FRANCE UNIVERSITY

Graph-Augmented Retrieval-Augmented Generation for Vietnamese Labor Law Question Answering

Thesis Report

Supervisor: Dr. Tran Hoang Tung

Student:

Tran Hien Chuong 22BA13056

Academic Year 2025–2026

Abstract

This thesis studies citation-grounded question answering for Vietnamese labor law. We focus on answers that users can verify through the cited document, article, clause, point, or appendix. The project was first motivated by employment contract termination, but the final scope covers a broader labor-law corpus because termination questions often depend on contract rules, retirement rules, minor-worker rules, and labor dispute procedures.

The system combines hierarchy-aware legal chunking, deterministic metadata enrichment, hybrid vector-sparse retrieval in Qdrant, and a Neo4j legal knowledge graph. The graph stores document structure, cross-references, and authority relations. At query time, hybrid retrieval provides seed contexts and graph expansion adds related legal provisions. The final candidate also uses intent gating and a deterministic scope guard before answer synthesis. If a query belongs to an unsupported topic family and the retrieved evidence is insufficient, the system returns an insufficient-context response instead of a legal answer. A rule-based validation layer checks that any generated answer cites only legal bases and article coordinates supported by the retrieved context.

We evaluated the system on six indexed Vietnamese labor-law and labor-procedure documents, divided into 1,556 legal chunks. The final benchmark contains 100 queries: 94 in-corpus queries with required citations and 6 out-of-corpus refusal tests. We found that graph expansion improves Recall@10 from 0.748 to 0.835, but increases the Forbidden Citation Violation Rate from 0.032 to 0.043. For the final graph-augmented retriever, Required Citation Coverage was also 0.835. The adjusted end-to-end pass rate over the full benchmark was 0.780. The answer pass rate was 0.970, the citation validation pass rate was 1.000, the quality validation pass rate was 0.970, and the out-of-corpus QA pass rate was 1.000. These results show that citation grounding and out-of-corpus refusal are strong in the benchmark, but in-corpus retrieval robustness remains limited.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Background

Information retrieval and question answering (QA) systems have become essential tools across many professional domains, including the legal sector. However, legal question answering differs fundamentally from general open-domain QA. In a legal context, an answer is only valid if it is grounded in the correct statutory text and accompanied by precise legal citations. Users—whether employees, employers, or legal advisors—require answers that preserve the specific hierarchical context of documents, articles, clauses, and points rather than simple semantic matches. Without accurate citations, an answer lacks legal authority and cannot be verified, which limits its practical utility for compliance or dispute resolution.

This research was originally motivated by a specific, high-frequency legal challenge: employment contract termination. Initially framed as "A Case Study on Employment Contract Termination," the project aimed to build an information assistant to guide users through the complex rules surrounding how labor relations end. Under Vietnamese law, termination disputes are highly sensitive, and incorrect administrative steps can lead to legal risk, financial consequences, or disputes. Providing accurate, citation-grounded answers for these scenarios is critical for both business compliance and employee advocacy.

However, during initial system design and testing, it became evident that contract termination cannot be treated as an isolated legal topic. A single inquiry about terminating an employment contract frequently depends on a wide range of broader labor-law provisions. For example, verifying whether a termination is lawful requires checking the underlying labor contract rules (such as contract types, probation periods, and mandatory contents) and the fundamental rights and obligations of both employees and employers. Determining the financial outcomes of termination requires traversing complex severance and compensation rules. Furthermore, special protection regimes, such as minor worker rules, and retirement age provisions, introduce distinct exceptions and tables that alter termination conditions. Finally, resolving these issues often involves analyzing labor dispute resolution and civil procedure provisions to determine court jurisdictions and mediation timelines.

Because of these interconnected legal dependencies, the scope of this project was expanded beyond a narrow, topic-specific case study. To support a broader scoped labor-law QA system, the system must cover the broader, structured network of Vietnamese labor law. This expanded corpus is composed of six key legislative and administrative documents: the Labor Code 2019, a labor-related subset of the Civil Procedure Code

2015, Decree 135/2020/NĐ-CP (governing retirement ages), Decree 145/2020/NĐ-CP (governing general labor conditions), Circular 09/2020/TT-BLĐTBXH (governing minor workers), and Circular 10/2020/TT-BLĐTBXH (governing contract templates).

Traditional information retrieval methods and standard Retrieval-Augmented Generation (RAG) pipelines [6] struggle with this hierarchical and cross-referenced complexity. Standard RAG architectures split documents into flat, fixed-size text windows that ignore the nested legal grammar. This arbitrary chunking frequently separates critical points from their parent articles or documents, causing semantic fragmentation. Moreover, standard RAG models operate primarily on semantic text similarity, meaning they can easily miss cross-referenced provisions, implementing decrees, circulars, and appendices that reside in different parts of the corpus.

Graph-augmented retrieval (or GraphRAG) provides a promising path forward by representing legal corpora as structural knowledge graphs. In this approach, nodes represent legal documents, articles, clauses, points, and appendices, while edges model the deterministic hierarchies, cross-references, taxonomic associations, and normative authority relations between them. By using vector-based semantic search to find initial "seed" chunks and then performing topological graph expansions, the system can dynamically retrieve related circulars, implementing decrees, and procedural rules. This integration of hybrid vector search and structured graph traversals helps provide the downstream generator with more relevant legal context, supporting more complete and better-grounded generated answers.

1.2 Problem Statement

Standard Retrieval-Augmented Generation systems typically rely on flat retrieval strategies. In these architectures, documents are processed into flat, sequential text blocks and stored in a vector database as isolated records. When a user asks a question, the retriever fetches a set of candidate chunks based solely on semantic vector similarity. While this approach works well for general knowledge-retrieval tasks, it introduces severe limitations in the legal domain, where statutory context is highly integrated.

Within the context of this project, a narrow termination-only dataset would be insufficient to support reliable question answering. Because legal rules are highly interdependent, many contract termination questions depend on broader legal provisions that lie outside the immediate topic of dismissal. For example, a user asking about a termination notice period might actually require a lookup in retirement age tables or contract-type definitions, and a question about wrongful dismissal remedies requires civil procedural rules to determine court jurisdiction. **The initial case study revealed the need for a broader scoped labor-law corpus.** Treating nested legal provisions as isolated, flat chunks can lead to incomplete answers. Under standard vector search, a ministerial circular that details a Labor Code article may not exhibit enough keyword or semantic overlap to be retrieved alongside its parent code, leaving the system with an incomplete representation of the relevant law.

Furthermore, relying on generative language models to write legal answers introduces the risk of citation hallucinations. If the inline citations generated in the final response are not rigorously validated, the model may cite unsupported provisions or reference incorrect article numbers, which is unacceptable in regulatory compliance.

Therefore, this project investigates how to combine vector retrieval, structured le-

gal graph expansion, and deterministic validation to improve citation-grounded Vietnamese labor-law question answering. To address these challenges, we need a system that integrates hierarchy-aware legal chunking to preserve statutory structures, hybrid vector-sparse retrieval to capture exact coordinates, graph expansion to navigate cross-references, and deterministic citation validation to verify that the cited legal basis and article coordinates are supported by the retrieved context. We aim to investigate these techniques within the indexed corpus and the constructed benchmark, without claiming correctness beyond this evaluation or suggesting that the software can replace human legal experts.

1.3 Research Objectives

To address the limitations of flat retrieval and citation hallucinations in legal QA, this research aims to design, implement, and evaluate a graph-augmented RAG pipeline for Vietnamese labor law. The primary objective is to investigate how combining hybrid vector search with topological graph expansion and rule-based validation affects the retrieval recall and citation grounding of generated answers within a scoped corpus.

Specifically, the project pursues the following concrete research objectives:

1. **Corpus Construction**: Build a scoped Vietnamese labor-law corpus consisting of six key legal and administrative documents, spanning different normative ranks, to support broad labor-law inquiries.

2. **Hierarchy-Aware Chunking**: Implement a syntax-aware chunking pipeline that parses legal texts at logical leaf-level boundaries (clauses or points) rather than arbitrary character lengths, preserving the document-to-article-to-clause-to-point structure.

3. **Metadata Enrichment**: Develop a rule-based enrichment layer to attach domain-specific metadata—including legal topics, targeted actors, issue types, document types, canonical citation texts, and legislative normative ranks—to each chunk.

4. **Hybrid Indexing**: Build a vector and hybrid retrieval index in a Qdrant database, utilizing dense SBERT embeddings and sparse lexical tokens to match semantic concepts and exact article coordinates concurrently.

5. **Knowledge Graph Design**: Construct a Neo4j legal knowledge graph that represents the structural hierarchies, resolved cross-references, rule-based taxonomic associations, and normative authority relationships within the corpus.

6. **Graph-Augmented Retrieval**: Implement a runtime retrieval expansion policy that uses Qdrant hybrid search hits as entry points (seed nodes) and traverses the Neo4j relational graph to gather parent contexts, implementing regulations, and coordinate fallbacks.

7. **Grounded Generation and Validation**: Establish an answering pipeline that generates citation-grounded responses and applies deterministic, rule-based validation checks to verify that the cited legal basis and article coordinates are supported by the retrieved context.

8. **System Evaluation**: Design a diagnostic evaluation setup and measure retrieval

recall, Required Citation Coverage, Forbidden Citation Violation Rate, and adjusted end-to-end pass rates over a constructed 100-query benchmark.

1.4 Scope of the Project

Defining the boundaries of this research is critical to establish the validity and limits of our findings. The project began as a case study on employment contract termination, focusing on the specific rules and disputes surrounding how labor relations end. However, because resolving contract termination inquiries requires broad statutory context, the final implementation was expanded to cover a broader Vietnamese labor-law question answering system. **The final scope should therefore be understood as a scoped Vietnamese labor-law QA system, expanded from the initial employment-contract-termination case study.**

The proposed system is limited to the six indexed Vietnamese labor-law and labor-procedure documents in our corpus. These documents comprise: the Labor Code 2019, a labor-related subset of the Civil Procedure Code 2015, Decree 135/2020/NĐ-CP, Decree 145/2020/NĐ-CP, Circular 09/2020/TT-BLĐTBXH, and Circular 10/2020/TT-BLĐTBXH. The system's knowledge and retrieval capabilities do not extend to other Vietnamese legislation, administrative guidelines, or regulatory domains.

The system is designed for legal information assistance and compliance guidance within the boundaries of this indexed corpus. It acts as an automated tool to help users navigate and locate relevant statutory passages, coordinate tables, and procedural rules. The system's output should not be treated as professional legal advice. Legal inquiries often involve subjective scenarios, changing regional practices, or unwritten judicial precedents that cannot be captured by an automated information retrieval system.

Furthermore, the evaluation of the system is conducted using a software-level, deterministic, rule-based validation framework over a controlled benchmark of 100 diagnostic queries. The benchmark contains 94 in-corpus queries with required citations and 6 out-of-corpus refusal tests. This project does not use human legal expert reviews as the final evaluation method to verify legal correctness in a live environment. Similarly, it does not rely on LLM-as-Judge metrics as the final measure of answer quality, because neural evaluators introduce stochastic variation and their own potential biases. Instead, all performance metrics, including retrieval recall, Required Citation Coverage, out-of-corpus refusal, and adjusted end-to-end pass rates, are computed programmatically against benchmark annotations within the scoped corpus. This deterministic approach supports reproducibility and transparent system verification, but it remains a software-level evaluation.

1.5 Contributions

This thesis contributes a modular, reproducible software framework for citation-grounded question answering in the Vietnamese legal domain. First, it builds a structured Vietnamese labor-law corpus from six core labor-law and labor-procedure documents. The corpus is processed through a hierarchy-aware legal chunking pipeline that identifies chapter, article, clause, and point boundaries, so that retrieved chunks remain aligned with the physical document structure. It also applies rule-based metadata enrichment to attach legal topics, targeted actors, issue types, normative ranks, and canonical citation

strings to each chunk.

Second, the thesis implements a retrieval architecture that combines hybrid indexing and retrieval with a Neo4j legal graph. The Qdrant indexing setup fuses dense Sentence-BERT embeddings with sparse lexical tokens and coordinate metadata, enabling semantic and exact-coordinate retrieval. The legal graph models structural hierarchies, cross-document reference links, rule-based taxonomy tags, and normative validity rankings. At runtime, the graph-augmented retrieval engine uses hybrid retrieval hits as entry points and traverses the graph to gather parent articles, implementing regulations, and related procedural guidelines.

Finally, the system connects retrieval to citation-grounded answer generation and validation. The answering pipeline combines extractive and generative components, then applies deterministic validation to enforce citation and coordinate grounding. The thesis also contributes a diagnostic benchmark and evaluation framework: a constructed benchmark of 100 diagnostic queries categorized by difficulty and functional type, together with reproducible scripts for measuring retrieval, citation grounding, answer quality, out-of-corpus refusal, and adjusted end-to-end pipeline pass rates.

1.6 Thesis Structure

The remainder of this thesis is organized into six chapters. Chapter 2 reviews the theoretical foundations of Large Language Models (LLMs), Retrieval-Augmented Generation (RAG) architectures, domain-specific legal question answering, knowledge graphs for information retrieval, graph-augmented RAG techniques, and evaluation methods. Chapter 3 presents the system design and methodology, including the offline ingestion pipeline, hierarchy-aware chunking, metadata enrichment, Neo4j graph schema, hybrid retrieval formulation, query routing, and graph expansion policies. Chapter 4 describes the implementation, covering the technology stack, Python data engineering scripts, Neo4j containerization, Qdrant indexing, Cypher-based graph traversal, answer validation gates, and reproducibility setup. Chapter 5 reports the empirical evaluation, including the 100-query benchmark, retrieval metrics, adjusted end-to-end pass rate, out-of-corpus refusal results, and representative case studies. Chapter 6 discusses the findings, strengths, failure modes, limitations, and deployment considerations. Chapter 7 concludes the thesis and outlines future research directions, including broader statutory coverage and expert-validated evaluation datasets.

Chapter 2

Background and Related Work

2.1 Large Language Models

Large Language Models (LLMs) represent a major advancement in natural language processing. These models, which are generally based on the transformer architecture, are pre-trained on massive collections of text to learn statistical representations of language structure, syntax, and semantics. Through this pre-training, LLMs acquire general capabilities in text generation, question answering, and text summarization, allowing them to process natural language inputs and synthesize coherent responses across various domains.

In domain-specific applications, LLMs are frequently utilized to interpret user queries and generate conversational responses. When applied to question answering, the model leverages its internal parametric memory—the associations and facts encoded within its weights during training—to construct answers. This capability allows models to translate informal, conversational user questions into structured responses or intermediate representations, making them useful interfaces for information access.

However, in legal question answering, relying solely on an LLM's parametric memory introduces severe limitations. The primary challenge is hallucination, where the model generates text that is linguistically plausible but legally incorrect or entirely fabricated. Furthermore, LLMs frequently experience context limits, lack access to updated or specialized domain knowledge, and struggle to produce accurate, verifiable legal citations. Because statutory rules are highly sensitive, a minor error in an article number or coordinate can alter the legal meaning, creating significant compliance risks.

Therefore, legal question-answering systems should not rely only on model memory to formulate answers. An LLM cannot be treated as a legal oracle. This limitation motivates the need for retrieval-based grounding. Rather than forcing the language model to retrieve statutory provisions from its static parameters, the system should locate verified legal passages from an external, structured corpus and present them to the model, ensuring that answer generation is grounded in retrieved legal sources.

2.2 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) is a framework designed to improve the reliability of language models by combining an external document retriever with a generative network [6]. The baseline workflow involves an offline indexing stage and an online query

stage. During indexing, a target text corpus is divided into small passages, converted into continuous vector representations using an embedding model, and stored in a vector database. At runtime, the system embeds the user's query, retrieves the most semantically similar passages, and assembles them into the model's prompt context to guide the generation layer.

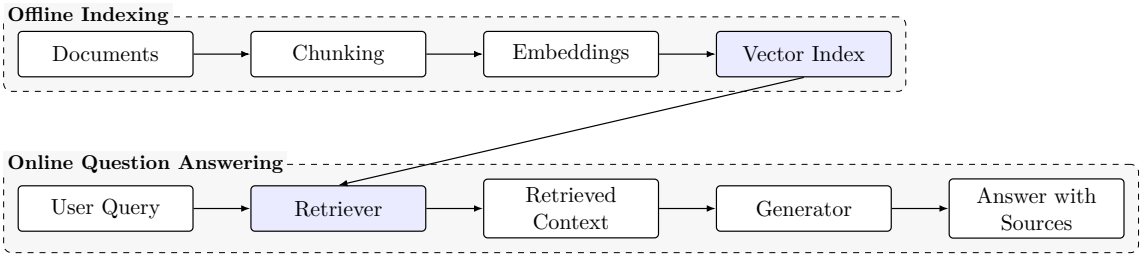

Figure 2.1: Standard retrieval-augmented generation pipeline.

The baseline RAG pipeline, illustrated in Figure 2.1, provides several advantages over standard generative answering. By supplying the generator with retrieved source passages, the framework reduces the risk of factual hallucinations, enables source-grounded answers, and provides a clear mechanism for citation tracing. Furthermore, RAG allows system administrators to update the underlying knowledge base dynamically by modifying the vector database, bypassing the expensive and complex training cycles required to retrain LLM parameters.

However, standard flat RAG architectures exhibit major limitations when applied to legal document corpora. In a flat RAG pipeline, documents are split into independent, fixed-size text blocks, which destroys the nested structure of statutory texts. Because flat retrieval treats these chunks as isolated text blocks, it is blind to parent-child hierarchies and cross-document reference networks. For example, standard flat RAG frequently separates a specific clause from its parent article title or document name, causing semantic fragmentation that makes the chunk uninterpretable for both the embedding model and the downstream generator.

Furthermore, relying solely on semantic vector similarity can lead to incomplete or noisy results. Dense embeddings excel at identifying general conceptual synonyms, but they frequently struggle to match exact article coordinates, numerical coordinate tables, or specific cross-referenced implementing decrees. If an essential statutory exception or guiding circular is ranked low by the vector search, it is omitted from the context window, preventing the generator from producing a legally complete answer. This challenge indicates the need for retrieval strategies that actively preserve and navigate legal document structures.

2.3 Legal QA Challenges

Legal question answering differs fundamentally from open-domain QA because legal answers require strict authority, jurisdiction specificity, temporal validity, and cautious wording [11]. A legal assertion is difficult to verify if it is not grounded in the correct legal basis and supported by precise citation traceability. Users need to verify the exact document, article, clause, and point that establishes a right or obligation. A system that summarizes general legal concepts without identifying these statutory anchors cannot be used for compliance or dispute resolution.

A primary challenge in legal QA is the nested, hierarchical structure of legislative texts. In Vietnam, statutory law is strictly organized by document, chapter, section, article, clause, point, and appendix, a structure governed by the Law No. 80/2015/QH13 on Promulgation of Legislative Documents [7]. To interpret a single legal clause correctly, the system must maintain its structural parent-child hierarchy and align it with its legislative normative rank. Under the principle of hierarchical authority, higher-level primary codes establish broad rights, while secondary administrative decrees and tertiary circulars detail their execution. A lower-level circular must remain strictly compliant with its parent decree and code, and conflicts are resolved in favor of the higher-ranking document.

This structural complexity is clearly illustrated by the motivating case study of this thesis: employment contract termination. The project was initially motivated by the specific rules and disputes surrounding how labor relations end. However, during system development, it was discovered that contract termination questions cannot be resolved in isolation. Resolving termination inquiries naturally requires broad statutory context, including labor contract rules (probation and contract types), employee and employer rights, unlawful termination criteria, severance and compensation calculations, minor worker rules, and female worker protections. Furthermore, retirement age tables in specialized decrees determine when a contract terminates due to retirement, and civil procedure rules in separate codes govern how disputes are litigated in court.

Consequently, the project's scope was expanded from a narrow, topic-specific case study to a broader Vietnamese labor-law QA system over six indexed documents. This transition highlights a key domain challenge: a legal QA system must retrieve not only semantically similar text but the exact, structured network of related legal provisions and guiding regulations. Benchmarks such as LexGLUE [1] and LegalBench [4] show the growing importance of evaluating legal language understanding and legal reasoning systems, although this thesis focuses on a smaller Vietnamese labor-law corpus with citation-level retrieval requirements.

2.4 Knowledge Graphs and Graph-Augmented RAG

Knowledge graphs (KGs) represent structured information by modeling entities and their semantic relationships as directed networks. In natural language processing, knowledge graphs are highly effective at representing complex, relational data that vector embeddings alone cannot capture. By storing data as nodes and typed edges, KGs allow systems to execute graph-traversal queries to discover non-obvious paths and structural dependencies across a text corpus.

In this thesis, the knowledge graph is not a stochastic, LLM-generated entity graph; it is a deterministic, document-structure legal knowledge graph. Rather than using generative models to auto-extract loose semantic entities—which introduces significant noise, missing links, and hallucinations—the graph is built programmatically using stable document nesting, regular-expression-based citation patterns, rule-based taxonomy metadata, and official normative ranks. This design choice ensures that the graph remains more reproducible and easier to inspect.

Graph-augmented retrieval (GraphRAG) combines the advantages of vector retrieval and structured knowledge graphs to address the limitations of flat RAG. In this architecture, vector and hybrid searches in Qdrant are utilized to locate initial, high-relevance seed

chunks. Starting from these entry points, the retriever executes parameterized Cypher queries in Neo4j to traverse structural parent edges, cross-document reference links, and taxonomic associations. This topological expansion retrieves parent articles, related circulars, implementing decrees, tables, and procedural guidelines that lack direct semantic similarity with the user's initial query, enriching the generator's context window.

This approach exhibits clear distinctions from the GraphRAG framework proposed by Microsoft [2]. The Microsoft GraphRAG system is designed for corpus-level global summarization, using LLMs to cluster text units into hierarchical communities and generate text summaries for each community. While effective for synthesizing broad narratives, this design is less directly aligned with the citation-level retrieval requirements of this thesis. In contrast, this thesis designs a smaller, deterministic, high-precision legal graph configured for citation-grounded retrieval and local context expansion over labor-law documents, prioritizing exact statutory coordinate matching over global corpus summarization.

2.5 Critical Comparison and Research Gap

The reviewed work shows that general RAG, legal NLP benchmarks, graph-based retrieval, and RAG evaluation frameworks each address part of the problem, but none of them directly covers the full setting of this thesis. Standard RAG is useful for grounding generation in retrieved passages, but it is weak when the answer depends on legal hierarchy and exact citation grounding. A flat chunk may contain a useful clause while losing the parent article, document title, or implementing provision that gives the clause its legal meaning. Dense similarity also does not reliably recover exact article numbers, clause coordinates, or cross-referenced legal bases.

Microsoft-style GraphRAG addresses a different retrieval problem. It uses graph communities and generated summaries to support global understanding and corpus-level synthesis [2]. This is valuable when the user asks broad questions over large unstructured collections. However, the present thesis needs provision-level retrieval: the system must return the specific article, clause, point, or appendix that supports an answer. For this reason, the graph in this work is not mainly a community-summary graph. It is a deterministic legal structure graph used to expand from retrieved seed chunks to related provisions and citations.

Legal NLP benchmarks such as LexGLUE [1] and LegalBench [4] are important because they show how legal language understanding can be evaluated across multiple tasks. However, they do not directly evaluate Vietnamese labor-law provision retrieval with article and clause citations. They also do not test whether a QA system retrieves the exact Vietnamese statutory basis needed for a user-facing answer. Therefore, this thesis uses a smaller diagnostic benchmark that is aligned with the indexed Vietnamese labor-law corpus and its citation requirements.

RAG evaluation frameworks such as Ragas [3] and ARES [10] motivate the separation of retrieval, answer quality, and faithfulness evaluation. At the same time, this thesis avoids LLM-as-Judge as the final evaluation method. In this legal citation setting, validation should be reproducible: the same answer and retrieved context should produce the same citation decision. A stochastic judge may be useful for exploratory analysis, but it is less suitable as the final source of benchmark metrics when article and clause validation can be checked with deterministic software rules.

The research gap addressed by this thesis is therefore a practical one: citation-grounded QA for a scoped Vietnamese labor-law corpus where the system must retrieve and validate exact legal bases. The thesis differs from prior work by combining Vietnamese labor-law hierarchy-aware chunking, Qdrant hybrid retrieval, Neo4j provision-level graph expansion, and deterministic citation validation. This combination is designed for traceable legal information access, not for broad legal reasoning or proof of legal correctness.

2.6 Evaluation Methods for RAG Systems

Evaluating Retrieval-Augmented Generation systems requires measuring multiple system components independently. RAG evaluation frameworks typically separate overall performance into three core dimensions: retrieval quality (assessing the relevance of retrieved context), answer faithfulness or quality (evaluating semantic correctness and naturalness), and citation grounding (checking whether generated claims match source text). Prior RAG evaluation work such as Ragas [3] and ARES [10] motivates separating retrieval quality, answer faithfulness, and answer relevance using automated neural evaluators.

In information retrieval, retrieval quality is quantitatively measured using citation-based metadata matching. Standard metrics include Recall@k (measuring the proportion of ground-truth citations recovered), Required Citation Coverage (verifying the complete retrieval of all gold citations), and the Forbidden Citation Violation Rate (measuring the frequency of retrieving obsolete or prohibited documents). Prioritizing high recall and citation coverage is essential in legal QA because missing a mandatory statutory provision prevents downstream generation from producing a complete and well-grounded answer.

For answer and citation evaluation, standard pipelines often rely on human legal expert reviews or LLM-as-Judge frameworks. However, in high-stakes regulatory settings, human expert review remains prohibitively expensive, and using stochastic language models as judges introduces prompt dependencies, high computational costs, and potential evaluation-level hallucinations. These limitations make stochastic evaluation difficult to reproduce or audit during iterative developer testing.

To ensure high reproducibility, transparency, and safety within a controlled environment, this thesis implements a simpler, deterministic, rule-based software validation setup rather than relying on human legal expert reviews or LLM-as-Judge as the final evaluation methods. This deterministic validation engine uses set-theoretic containment and regular expression parsing to programmatically verify that generated citations and article numbers are supported by retrieved contexts. Although this software-level verification is highly reproducible and useful for identifying structural errors, it does not replace legal expert judgment and may miss subtle semantic nuances or complex interpretative disagreements.

Chapter 3

System Design and Methodology

3.1 Overall System Architecture

Figure 3.1 illustrates the overall architecture of our graph-augmented Retrieval-Augmented Generation (GraphRAG) system, which is designed to support citation-grounded answers for Vietnamese labor-law questions. The main goal of this system is to retrieve accurate legal evidence from the indexed corpus and generate citation-grounded answers from retrieved evidence.

A standard, flat RAG pipeline often fails in the legal domain because it misses crucial context. Vietnamese legal documents have a highly nested structure, typically organized as Document → Article → Clause → Point. A simple flat retrieval strategy splits these texts arbitrarily, which separates detailed clauses from their parent articles or documents. Furthermore, answering a single legal question often requires retrieving related decrees, circulars, appendices, or cross-referenced provisions that reside in different parts of the corpus. To preserve this legal structure and connect vector retrieval with graph expansion, our architecture combines semantic vector search with a legal knowledge graph.

The system is divided into two primary parts: an offline data processing pipeline and an online query-answering pipeline.

The offline pipeline begins by processing raw legal documents into hierarchy-aware chunks. It first normalizes unicode characters and removes OCR noise, headers, footers, and page numbers from the source files. After the text is normalized, the parser identifies legal structure boundaries, such as chapters, articles, clauses, and points, using regular expression patterns. The parser then splits each document at leaf-level legal units, such as clauses or points, rather than using arbitrary word counts. This hierarchy-aware chunking reduces semantic fragmentation. Each chunk is then formatted through a semantic preservation layer, which makes leaf-level chunks inherit metadata such as document title, article number, and normative rank from their parent elements. Finally, the pipeline attaches domain taxonomy metadata, including legal topics, actors, and issue types, and extracts relative and document-level cross-references mentioned within each chunk.

Once the chunks are processed, they are indexed in two complementary knowledge stores: Qdrant and Neo4j. We use Qdrant to store and query dense semantic and sparse lexical embeddings for hybrid vector retrieval. Simultaneously, we load the document structures, metadata, and cross-references into Neo4j to build a legal knowledge graph. A shared `chunk_id` serves as the primary key that links the vector records in Qdrant with the corresponding evidence nodes in Neo4j, allowing us to connect vector retrieval with graph expansion at runtime.

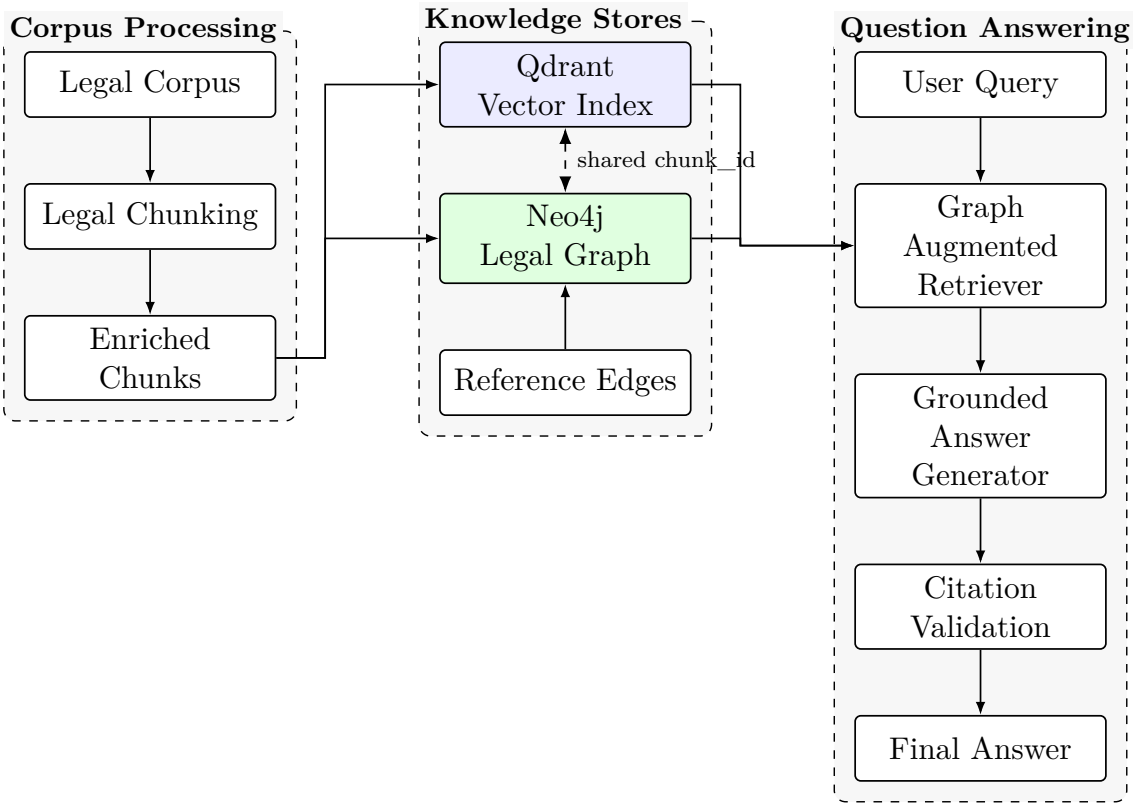

Figure 3.1: Overall architecture of the proposed graph-augmented RAG system.

At runtime, the online question-answering pipeline starts from a user query q written in natural language. The query routing layer identifies target topics, actors, and specific article references. The system then performs hybrid seed retrieval in Qdrant, combining dense semantic search with sparse keyword search to find the initial seed chunks. These seed chunks become entry points for graph and reference expansion in Neo4j, where the retriever gathers parent articles, related circulars, and implementing decrees. The seed and graph-expanded chunks are then deduplicated, reranked, and assembled into the final context window. This context is passed to the Large Language Model (LLM) for grounded answer generation, after which citation validation checks whether the draft answer and inline citations are supported by the retrieved evidence. The pipeline finally returns a citation-grounded response, or an explicit insufficient-context message when the retrieved sources do not support the query.

To formalize the context assembly step, the final context array C_{final} loaded into the generation layer is defined as:

$$C_{\text{final}} = \text{Rerank}\left(S_k \cup C_{\text{graph}} \cup C_{\text{fallback}}\right),$$

where S_k represents the set of seed chunks retrieved via hybrid search, C_{graph} denotes the additional context gathered through the Neo4j graph expansion, and C_{fallback} represents the coordinate and reference fallback contexts retrieved when specific article numerals are explicitly mentioned in the query.

After the generation step, the draft response passes through an answer validation layer. Instead of assuming the LLM's output is always legally correct, this layer uses the retrieved citation surface and applies deterministic checks to reduce unsupported answers. The validator verifies that all generated citations appear in the retrieved chunks, that

generated article numbers are supported by the source text, and that higher-rank legal contexts, such as Code provisions, are not ignored or contradicted by lower-rank circulars. It also activates an insufficient-context disclosure when the retrieved documents do not contain enough evidence to answer the query. This validation process helps prevent hallucinations and keeps the generated answers grounded within the indexed corpus, without claiming universal legal correctness.

The subsequent sections of this chapter describe each of these components in detail. Section 3.2 outlines the characteristics of our legal corpus and metadata. Section 3.3 covers the document preprocessing and hierarchical chunking techniques. Section 3.4 explains the design and schema of the Neo4j legal knowledge graph. Section 3.5 formalizes our vector-based and hybrid retrieval strategies. Section 3.6 details the runtime online query routing and graph expansion policies. Finally, Section 3.7 describes the context reranking, budgeting rules, and the deterministic validation guardrails.

3.2 Legal Corpus and Data Sources

The performance, reliability, and precision of a Retrieval-Augmented Generation (RAG) system are fundamentally governed by the scope, structural integrity, and authority of its underlying knowledge base. In the domain of legal question answering, these parameters are especially critical: legal language is highly structured, dense, and semantically sensitive, where minor phrasing variations carry distinct statutory implications. Consequently, retrieval failures or LLM-based hallucinations in legal contexts can lead to incorrect advice, regulatory non-compliance, or severe procedural errors. To mitigate these risks and establish a verifiable QA pipeline, this study utilizes an intentionally scoped legal corpus consisting of six key Vietnamese labor-related documents, summarized in Table 3.1.

Rather than attempting to index the entire, dynamic repository of Vietnamese national legislation—which spans tens of thousands of laws, administrative decrees, and circulars across disjoint regulatory domains—this project deliberately restricts its scope to a representative core of labor law. This intentional scoping serves three main methodological purposes: First, it provides a controlled environment for evaluating the extraction accuracy of legal cross-references and parent-child hierarchical relations, which are critical for graph-based semantic indexing. Second, this controlled scope allows the evaluation to focus on citation coverage, deterministic validation, and reproducible retrieval behavior within the indexed corpus. Third, it directly mirrors the architecture of domain-specific legal expert systems, which prioritize deep, specialized semantic coverage over broad but shallow general knowledge. It is important to emphasize that the proposed system is bounded by this indexed corpus and does not encompass all Vietnamese legislative texts. It is designed as a domain-specific information assistant for targeted labor-law inquiries, not as a general legal oracle or a tool for live legal decision-making.

Table 3.1: Summary of the scoped Vietnamese labor law corpus and document metadata.

Document ID	Type	Normative Rank	Chunks
`45-2019-qh14`	`bo_luat`	1	697
`92-2015-qh13-labor-only`	`bo_luat`	1	159
`nghi-dinh-135-2020-nd-cp`	`nghi_dinh`	2	41
`nghi-dinh-145-2020-nd-cp`	`nghi_dinh`	2	520
`thong-tu-09-2020-tt-bldtbxh`	`thong_tu`	3	73
`thong-tu-10-2020-tt-bldtbxh`	`thong_tu`	3	66

A defining characteristic of the Vietnamese legal framework is its strict hierarchical structure of legal validity, which maps directly to the normative ranks defined in Table 3.1. This hierarchical alignment is governed by the national Law on Promulgation of Legislative Documents [7]. Understanding this hierarchy is essential for both query routing and context synthesis, as lower-level regulations must conform to higher-level statutes, and conflicts are legally resolved in favor of the higher-ranking document.

Normative Rank 1 (Codes and Laws). This rank represents the highest level of domestic statutory authority, containing primary legislative acts passed directly by the National Assembly of Vietnam. In our corpus, these are represented by the type `bo_luat` (code). Under the principle of *lex superior*, these documents establish the foundational legal rights, duties, and frameworks that all subordinate instruments must strictly adhere to.

Normative Rank 2 (Decrees). Promulgated by the Government of Vietnam and signed by the Prime Minister, decrees (represented by the type `nghi_dinh`) occupy the second level of the hierarchy. The primary function of a decree is to detail, guide, and implement the broad provisions outlined in a Rank 1 code. While subordinate to codes, decrees carry substantial administrative authority and represent the main operational directives for government ministries and public enforcement.

Normative Rank 3 (Circulars). Positioned at the third level of the hierarchy, circulars (represented by the type `thong_tu`) are issued by individual ministries or ministerial-level agencies. In this corpus, these documents are promulgated by the Ministry of Labour, Invalids and Social Affairs (MOLISA) to provide detailed procedural instructions, administrative forms, and narrow technical specifications required to execute decrees and codes. Under Vietnamese legal practice, circulars must remain strictly within the bounds of their parent codes and decrees.

To construct a comprehensive representation of labor relations and disputes, the selected six documents cover the entire span of the legal hierarchy—from primary statutory rights down to specific administrative templates and procedural litigation rules:

Labor Code 2019. Represented by the identifier `45-2019-qh14`, the Labor Code 2019 is the foundational legislative pillar of the entire corpus. Promulgated by the National Assembly on November 20, 2019, and taking effect on January 1, 2021, it establishes the fundamental principles governing labor relations, employment standards, trade unions, social dialogue, and labor dispute resolution. The document comprises 17 chapters and 220 articles, which are preprocessed into 697 distinct text chunks. It acts as the ultimate authority for defining employment contracts, statutory benefits, minimum working age, and occupational rights.

Implementing decrees and circulars. To model the detailed administrative and

procedural rules that implement the Labor Code 2019, the corpus integrates four key guiding documents:

- *Decree 145/2020/ND-CP* (`nghi-dinh-145-2020-nd-cp`): This Rank 2 administrative decree is a massive regulatory document (yielding 520 chunks) that provides comprehensive guidance on several core aspects of the Labor Code, including specific regulations on labor contract conditions, wage calculations, overtime hours, labor discipline, occupational safety, and mandatory policies for female and minor employees. It represents the primary source of operational details for daily business compliance.
- *Decree 135/2020/ND-CP* (`nghi-dinh-135-2020-nd-cp`): This specialized Rank 2 decree establishes the regulatory roadmap for increasing the national retirement age in Vietnam. It contains the official retirement age lookup tables and scheduling indexes based on the employee's gender, birth month, and work hazards, preprocessed into 41 chunks.
- *Circular 09/2020/TT-BLDTBXH* (`thong-tu-09-2020-tt-bldtbxh`): Promulgated by MOLISA, this Rank 3 circular (yielding 73 chunks) details the protective regulations and specific conditions governing employment relations with minor workers under 15 years of age. It includes the official list of permissible light-work occupations and mandatory health-and-safety guidelines for employing youth.
- *Circular 10/2020/TT-BLDTBXH* (`thong-tu-10-2020-tt-bldtbxh`): This Rank 3 circular (yielding 66 chunks) provides detailed templates and mandatory content requirements for labor contracts, covering job scopes, wage structures, social insurance obligations, and intellectual property clauses.

Labor-related civil procedure provisions. Represented by the document ID `92-2015-qh13-labor-only`, this dataset consists of a carefully filtered subset of the Civil Procedure Code 2015 (Law No. 92/2015/QH13), yielding 159 chunks. While the complete Civil Procedure Code governs all civil, commercial, and family lawsuits in Vietnam, our corpus selectively extracts only the specific provisions that govern labor disputes, court jurisdictions, evidentiary requirements, litigation fees, and procedural timelines for labor-related lawsuits. The inclusion of these Rank 1 provisions is vital for query answering because employment-related questions frequently transition from substantive rights (e.g., "What are my rights under unfair dismissal?") to procedural remedies (e.g., "How and where do I file a lawsuit against my employer?"). By linking litigation procedures directly to substantive labor rights, the corpus enables the QA system to support multi-hop legal reasoning and comprehensive user guidance.

3.3 Legal Document Preprocessing and Enrichment

The performance of a Retrieval-Augmented Generation (RAG) system in the legal domain depends heavily on how raw documents are processed into structured, queryable units. Legal texts use a strict hierarchical organization that is crucial for interpreting statutory provisions. In Vietnam, legislation follows a deeply nested structure of chapters, sections, articles (*Điều*), clauses (*Khoản*), and points (*Điểm*). Standard RAG pipelines typically split documents using flat, fixed-size sliding text windows. However, this arbitrary segmentation is unsuitable for legal texts because it is blind to the structural grammar. It

frequently splits cohesive provisions across chunk boundaries, causing semantic fragmentation and separating nested points from their parent clauses and articles. For example, a retrieved chunk containing only "*c) Người từ đủ 15 tuổi đến dưới 18 tuổi*" (c) Persons from full 15 to under 18 years of age) is uninterpretable without knowing this defines the age range for minor employment under Article 143. Naive chunking destroys the context necessary for dense vector matching and makes it difficult for an LLM to generate or validate precise citations. To address this, our system implements a specialized data processing pipeline (Figure 3.2) that maps raw texts into structured, metadata-enriched chunks.

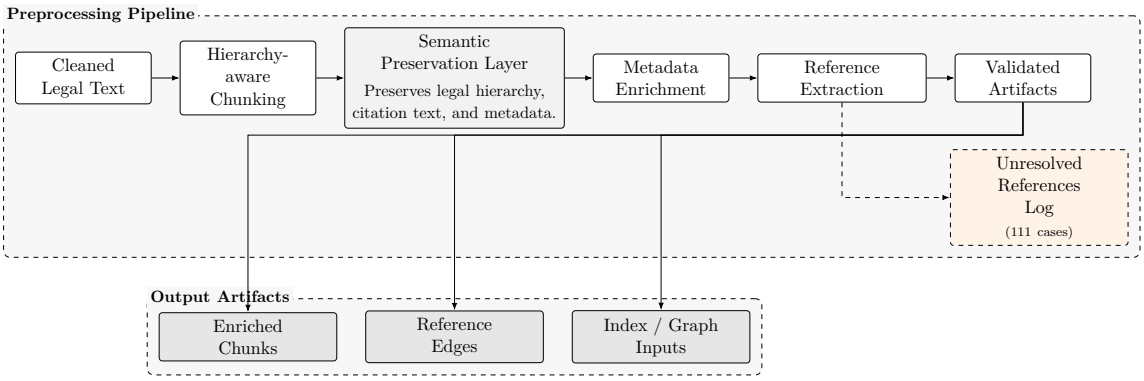

Figure 3.2: Legal data processing pipeline with hierarchy-aware chunking and metadata preservation.

Text cleaning. Before structural parsing begins, raw documents are converted into normalized Unicode (NFC) encoding to resolve character inconsistencies common in older PDFs. Document-specific headers, footers, page numbers, watermarks, and OCR artifacts are stripped. Double line breaks and indentations are normalized while maintaining paragraph borders, producing a clean, standardized text stream.

Hierarchical chunking. Rather than slicing the text stream at uniform intervals, our chunking engine uses a syntax-aware parser that isolates legal units using deterministic regular expressions. These rules identify legal boundaries and map the nested hierarchy into parented, object-oriented structures. The parser detects parts, chapters, sections, articles, clauses, points, and subpoints. The full regular-expression patterns are provided in Appendix B. The parser also handles subpoint patterns (such as a.1 or b.2) explicitly to prevent granular exceptions or conditions from being merged into generic point texts. This syntax-aware parser processes the document stream to build a lightweight hierarchical representation of nested legal provisions, preserving headings and layout relationships.

Semantic preservation. The core of our preprocessing pipeline is the Semantic Preservation Layer. This layer is designed to maintain the statutory context of nested legal provisions across chunk boundaries by using metadata inheritance. When the hierarchical chunker extracts a granular leaf-level unit, the Semantic Preservation Layer forces that chunk to inherit structural and citation metadata from its parent clause, article, and document. For example, consider the concrete legal provision *Điểm c, Khoản 2, Điều 3* of the Labor Code 2019. Under naive chunking, this point is isolated as a flat string. Under our Semantic Preservation Layer, the resulting chunk inherits the point, clause, article number, document identifier, document title, and citation surface. The exact implementation field names are documented in Appendix B. This metadata inher-

itance supports downstream operations by enabling vector searches to be combined with metadata filtering and providing the LLM with the exact legal basis for its statements, making citation generation easier to verify within the indexed corpus.

To optimize both search accuracy and response generation, the pipeline splits each chunk's textual content into two distinct fields: `retrieval_text` and `citation_text`. The retrieval field is designed to make semantic vector retrieval more reliable. It combines the raw text of the chunk with parent hierarchical context headers, such as the document title and parent article title. This context injection provides the embedding model with the complete legal framework, helping it calculate more accurate similarity scores even when user queries are implicit or abstract. Conversely, the citation surface represents the exact, formal legal location, for example a point, clause, article, and document title. This split gives each field a clear role: the retrieval field supports matching during vector search, while the citation surface is used for citation grounding and reduces the risk of hallucinations in LLM responses.

Table 3.2: Legal chunking and data integrity validation results.

Validation Item	Value
Legal chunks	1,556
Duplicate chunk IDs	0
Missing citation text	0
Missing article number	0
Very short chunks	0
Very long chunks	0

Metadata enrichment. Beyond structural attributes, the pipeline enriches each chunk with metadata for indexing and query routing, including its identifier, type, and source file. The enrichment pipeline uses a deterministic, rule-based mapping function, `infer_enriched_taxonomy`, to assign legal taxonomy categories. This was a concrete engineering decision: stable metadata is needed for reproducible indexing and for debugging retrieval failures. The function builds a normalized aggregated context string from the chunk's hierarchical titles and text, then scans it using the rule dictionaries `TOPIC_RULES`, `ACTOR_RULES`, and `ISSUE_TYPE_RULES`. These dictionaries cover core legal topics, legal actors, and legal issue types. A rule-based dictionary scan also avoids stochastic metadata labels and keeps ingestion inexpensive.

Appendix and table handling. Unlike standard text processors that often struggle to parse non-paragraph elements and discard tables as noise, our pipeline treats appendices and tables as primary structured evidence chunks. This is important because critical legal details—such as the gender-specific retirement age lookup tables in Decree 135/2020/ND-CP or standard contract templates in Circular 10/2020/TT-BLDTBXH—are housed entirely within tables and appendices. Our parser converts these tables into dense markdown representations, preserving column relationships and ensuring that cell-level lookups are searchable. These chunks are embedded, indexed with table or appendix type metadata, and linked directly to their parent document nodes in the graph database, allowing the online system to retrieve exact lookup tables.

Validation. To verify the performance of the data processing pipeline before database ingestion, the corpus is passed through an automated validation layer. This layer performs structural sanity checks to detect duplicate identifiers, empty payloads, orphan nodes, and

anomalous chunk lengths. The pipeline uses a fixed pruning threshold of 20 characters to discard fragmented lines or parsing noise, making the validation process quantitative rather than subjective. This threshold is a trade-off. A higher threshold may remove legitimate short legal points, while a lower threshold may allow OCR fragments to remain.

As reported in Table 3.2, the preprocessing pipeline of the scoped corpus yielded exactly 1,556 validated legal chunks. The validation run completed with zero duplicate chunk IDs, zero instances of missing citation texts, zero missing article numbers, and zero short or long chunks. This outcome indicates that our pipeline is stable: the absence of duplicate chunk IDs shows consistent chunk identification, the zero missing citation text count indicates that every evidence chunk has a valid statutory address, and the lack of very short chunks shows that the parser filtered empty lines and parsing noise below the 20-character threshold without losing genuine legal content.

3.4 Legal Cross-Reference Extraction and Graph Design

Answering legal questions often requires navigating a complex web of cross-references and structural hierarchies. Legal provisions are rarely read in isolation. For instance, a clause in a ministerial circular is frequently qualified by an article in an implementing decree, which in turn derives its authority from a primary code. To capture these structural and thematic relationships, we represent our corpus as a directed legal knowledge graph stored in a Neo4j database. This graph is deterministic and document-structured. It is constructed directly from stable legal hierarchies, regular-expression-based citation patterns, explicit cross-references, rule-based taxonomy metadata, and normative legal validity ranks. It is not a stochastic, LLM-generated entity graph, which would be prone to missing links and hallucinated connections. Using a deterministic schema supports reproducible graph construction and makes the graph easier to inspect.

Graph schema. The conceptual schema of the Neo4j legal knowledge graph is illustrated in Figure 3.3. The graph is organized as a directed network of legal document, article, clause, point, appendix, evidence, topic, actor, and issue-type nodes. The exact Neo4j labels are provided in Appendix C. These nodes are connected by relationships that capture the structural, referential, taxonomic, and normative dimensions of the scoped Vietnamese labor-law corpus.

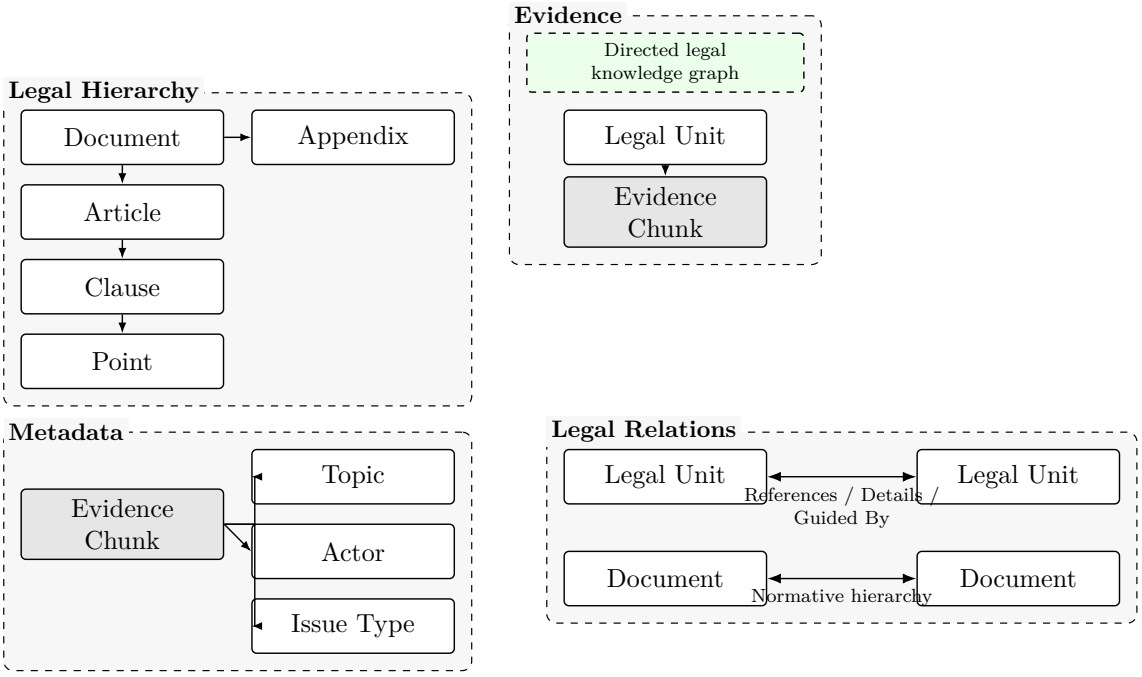

Figure 3.3: Conceptual schema of the Neo4j legal knowledge graph.

The schema organizes relationships by their retrieval function. Structural hierarchy relationships represent the internal skeleton of each document and map document nesting from documents to appendices. Source chunk and evidence links connect raw evidence chunks to leaf-level parent nodes, such as points, clauses, or articles. Cross-reference edges model citation links between legal provisions. The exact hierarchy and reference labels are shown in Figure 3.3 and Appendix C.

The remaining relationship groups support filtering and legal authority. Taxonomy edges link evidence chunks to semantic classification nodes for topics, actors, and issue types. Normative hierarchy edges represent legal authority between documents, such as Code → Decree → Circular. These relationships help the retriever apply metadata filters and prioritize higher-rank statutory authority.

Citation pattern detection. To build this directed legal network, the preprocessing pipeline uses a specialized citation parsing and reference resolution workflow. The extraction engine scans the raw text of each chunk using regular expressions designed to capture the varied citation styles in Vietnamese legislative documents. The parser identifies document-level citations, which name specific target documents (e.g., "Điều 5 Nghị định số 145/2020/NĐ-CP"), and relative citations, which refer to local or parent structures within the same document (e.g., "Khoản 2 Điều này"). To handle the syntactic complexity of Vietnamese legislative drafting, where multiple clauses are often coordinated in a single sentence, the parser implements a compound reference extractor. For example, a phrase like "khoản 1, 2 và 3 Điều 10" is not collapsed into a single edge; instead, the extractor parses the coordinated clause sequence and programmatically instantiates three separate target references to Clause 1, Clause 2, and Clause 3 of Article 10. This approach preserves graph granularity, so distinct legal citations can be traversed individually during retrieval.

Reference resolution. Once citation patterns are detected, they are passed to a normalization engine to resolve their target node identifiers. Relative citations are mapped to full node paths by looking up the parent article and document ID within

the structural hierarchy constructed during preprocessing. Document-level citations are normalized by matching document titles and abbreviations against a registry of aliases (e.g., mapping "Bộ luật Lao động" to the document ID `45-2019-qh14`). Localized relative references like "nghị định này" (this decree), "thông tư này" (this circular), or "luật này" (this law) are resolved dynamically using the source chunk's metadata context. A notable special case occurs in Article 219 of the Labor Code 2019, which contains transitional and amending provisions. In this article, the expression "Luật này" refers to various external amended laws (such as the Law on Social Insurance) rather than the Labor Code itself. The resolver handles this by mapping the target to a dedicated external amended-law placeholder node instead of forcing it into the internal structure of the Labor Code, which reduces the risk of incorrect edges.

Edge typing. When resolved references are loaded into the database, the edge type between a source node s and a target node t is computed dynamically based on their legislative normative ranks. Normative ranks are represented as numeric values in our document metadata. In this ranking, Rank 1 represents a primary Code or Law, Rank 2 represents an administrative Decree, and Rank 3 represents a ministerial Circular. Under this scheme, a larger numeric rank denotes a lower-level document in the legislative hierarchy (e.g., a Rank 3 circular is subordinate to a Rank 2 decree, which is subordinate to a Rank 1 code). The formal edge-typing rule is defined by the following mapping:

$$\text{EdgeType}(s,t) = \begin{cases} \text{DETAILS}, & \text{if } \text{Rank}(s) > \text{Rank}(t), \\ \text{GUIDED\_BY}, & \text{if } \text{Rank}(s) < \text{Rank}(t), \\ \text{REFERENCES}, & \text{otherwise.} \end{cases}$$

Under this rule, when a lower-level document cites a higher-level statute, such as a Rank 2 decree detailing a Rank 1 code provision, the edge is typed as a detail edge. When a higher-level statute points to a lower-level implementing regulation, the edge is typed as a guidance edge. Same-rank or neutral references are typed as reference edges.

Additionally, when a cross-document detail edge is generated, the pipeline automatically instantiates a corresponding inverse guidance edge. For example, since a decree or circular often details a provision in the Labor Code, the primary detail edge captures the citation direction from the lower-level text to the higher-level code. Creating the inverse guidance edge is a deliberate design choice that allows graph traversal to run in both directions: from implementing guidance back to the source law, and from the source law to its guiding regulations. This bidirectional capability helps graph expansion during retrieval by allowing the traversal engine to gather related documents regardless of which one contains the original citation link.

Unresolved references. To keep the knowledge graph clean, we do not assume all citations can be successfully resolved. Out of 1,059 citation expressions identified by the extraction engine within the scoped corpus, exactly 948 reference edges were resolved and loaded into the Neo4j knowledge graph, as summarized in Table 3.3. The remaining 111 references were logged and intentionally excluded from graph traversal, rather than being treated as system errors. This deliberate omission represents a conservative graph construction philosophy that keeps uncertain references out of the active graph. The 111 unresolved references primarily stem from references to external legislative documents outside our six-document corpus (such as the Civil Code 2015), incomplete textual coordinates, references to external laws, and ambiguous relative references that lacked specific anchor points. Excluding these uncertain references keeps the active graph conservative,

but it also limits graph expansion to references that can be resolved within the indexed corpus.

Table 3.3: Summary of loaded node and edge counts in the Neo4j legal knowledge graph.

Graph Item	Count
Documents	6
Articles	411
Clauses	1,235
Points	774
Appendices	35
Evidence chunks	1,556
Resolved reference edges loaded	948
Taxonomy edges	6,099
Normative hierarchy edges	20

3.5 Vector-based and Hybrid Retrieval

Before executing graph-based context expansion, the system establishes a baseline retrieval layer using vector-based and hybrid search. Relying on a single retrieval modality is often insufficient for legal question answering. While dense semantic search is highly effective at identifying conceptual synonyms, it can miss or be less effective for matching exact statutory citations or numerical coordinates. Conversely, sparse keyword matching is very precise at capturing exact article numbers and legislative IDs but is less effective for matching paraphrased conversational queries. To resolve these individual limitations, our system implements a unified hybrid retrieval framework that combines both approaches using the Qdrant vector database [8] to index and query our preprocessed legal corpus.

The baseline retrieval engine operates over the 1,556 hierarchy-aware legal chunks in our scoped corpus. During database ingestion, each chunk is encoded into both dense and sparse vector representations. The dense semantic embeddings are generated using the `keepitreal/vietnamese-sbert` model [9, 5], which is a Sentence-BERT transformer optimized for Vietnamese natural language processing. This model projects each text chunk into a 384-dimensional dense vector space. For sparse lexical matching, the chunk text is processed using a sparse encoder to generate term-importance vectors that capture structural keywords and statutory terminology. Each Qdrant vector entry also preserves payload metadata for filtering and routing, including chunk identity, document identity, article and clause coordinates, citation surface, document type, normative rank, topic, actor, and issue type. The exact payload field names are documented in Appendix B. Preserving this payload allows the system to apply legal intent-specific constraints during query execution.

At query execution time, the search engine does not simply calculate the similarity score of the raw user query. Instead, as shown in our overall pipeline, the query first passes through the routing layer to identify structural intents, documents, and coordinates. The system then generates multiple semantic query variations using the internal helper `build_query_variants`. This step helps bridge the lexical gap between a user's informal conversational question and the highly formal statutory phrasing of Vietnamese labor regulations. The generated set of query variants, V, can include the raw user query,

rule-based query expansions, issue-focused terms, and specific citation coordinates. For each query variant, the retrieval engine queries Qdrant using parallel prefetch channels that run both dense semantic and sparse keyword searches concurrently.

To reduce embedding misalignment and avoid relying on a single vector representation of the user query, the system prefetches candidates across multiple dense and sparse channels. The total number of parallel prefetch channels, $|P|$, executed for a single query is determined by the following relation:

$$|P| = 2|V| + \mathbb{I}(\text{ref\_boost}) + \mathbb{I}(\text{issue\_focus})$$

where V is the set of generated query variants, and P is the set of Qdrant prefetch channels. The term $2|V|$ represents the concurrent execution of one dense and one sparse prefetch channel for each generated query variant. The term $\mathbb{I}(\text{ref\_boost})$ is an indicator function that equals 1 when a reference-boost filter is active, and 0 otherwise. Similarly, $\mathbb{I}(\text{issue\_focus})$ is an indicator function that equals 1 when an issue-focus filter is active, and 0 otherwise.

In the implemented retrieval engine, both the reference-boost filter (ref_boost) and the issue-focus filter (issue_focus) are routed exclusively through the sparse vector prefetch channel. This is an important design choice: because article numbers, document IDs, issue types, and legal actors are represented as deterministic legal tokens, they are matched more reliably by sparse lexical indexing combined with payload filtering. Routing these high-precision boosts through the sparse vector channel rather than relying on dense embeddings helps reduce dense semantic noise and prevents exact legal coordinates from being blurred by superficial embedding similarities.

Once candidate lists are returned from all active prefetch channels, the search engine merges and reranks them using Reciprocal Rank Fusion (RRF). In our implementation, this fusion operation is delegated directly to Qdrant's built-in RRF operator. Conceptually, the RRF score for a candidate chunk d is computed by the following mapping:

$$\text{RRF}(d) = \sum_{m \in M} \frac{1}{k + r_m(d)}$$

where M represents the set of active prefetch channels, $r_m(d)$ is the ordinal rank of candidate d within prefetch channel m (with $r_m(d) = \infty$ if d is absent from the retrieval list of channel m), and k is a smoothing constant that helps prevent top-ranked items in any single channel from dominating the fused result. Because this fusion is handled natively by Qdrant's query engine, the formula above represents the underlying ranking principle, while the exact value of the smoothing constant k is managed by the backend search engine configuration.

Fusing dense semantic search with sparse lexical precision improves the chance of retrieving highly relevant chunks (such as exact articles like "Điều 219", or documents like "Nghị định 145" and "Thông tư 10") while maintaining broad semantic coverage. However, while hybrid retrieval provides a strong baseline retrieval layer, it still treats chunks as isolated evidence and remains oblivious to the structural and referential bonds between them. For example, if a primary code article (such as Article 143 on minor workers) explicitly references a ministerial circular for detailed occupational lists (such as Circular 09/2020/TT-BLDTBXH), a hybrid search can retrieve the Labor Code chunk but cannot dynamically traverse the relationship to fetch the circular's details unless the query itself explicitly contains those circular-specific terms. To answer complex multi-hop

queries, the system must navigate these statutory cross-references. In our architecture, the hybrid retrieval layer is used to provide a set of strong seed chunks for graph expansion.

3.6 Query Routing and Graph Expansion Policy

As discussed in Section 3.5, while hybrid search provides strong baseline seed chunks S_k, resolving complex legal questions requires traversing statutory cross-references, taxonomic connections, and legislative hierarchies. In our implementation, the query routing layer acts as the main routing component. It translates a raw, conversational user query into a structured, formal `QueryIntent` control object. This control object dynamically guides subsequent retrieval stages, including candidate seed pre-filtering, forced reference promotion, coordinate fallback, and adaptive context windowing. These steps enrich the retrieval context before generation. Specifically, query routing resolves the legal intent and constraints, which enables graph-based reference expansion to fill the missing legal context.

Intent resolution. The query routing layer executes a dual-stage routing process consisting of a structured, language-model-based metadata extractor to resolve semantic intent, and a deterministic validation and expansion layer to align queries with the statutory domain. To extract structural intent, a structured language model acts as a metadata extractor. This model structures the query's legal context into fields for actors, topics, issues, document identifiers, query types, article numbers, clause references, and point references. The model is configured with temperature set to 0 to minimize stochastic variance. The model is used only for metadata extraction, not for generating legal answers, which reduces the risk of generating inaccurate or hallucinated statutory text.

Metadata sanitization. Once the structured metadata payload is returned, the system routes it through a cleaning step that normalizes and filters all extracted metadata against valid domain vocabularies, such as valid actors, topics, issues, document IDs, and query types. This sanitization layer reduces invalid metadata by discarding out-of-vocabulary terms. Formally, for any extracted metadata collection, the sanitization operation resolves valid labels through the following mapping:

$$L_{\text{valid}} = \{\, l \in L_{\text{extracted}} \mid \text{Normalize}(l) \in \Omega \,\}$$

where $L_{\text{extracted}}$ is the set of raw labels extracted by the language model, Ω represents the corresponding set of valid domain labels, and the function Normalize(l) represents a deterministic unicode and casing normalization pipeline. Casing is standardized to lowercase, accents are stripped, and spaces are mapped to underscores. Discarding invalid labels prevents stochastic errors from entering the retrieval pipeline.

QueryIntent construction. After the metadata is sanitized, the system constructs a unified `QueryIntent` control object. Rather than relying solely on the language model's classification, the system dynamically merges the sanitized metadata with keyword matches extracted directly from the normalized query stream, rule-based routing coordinates, and mapped article coordinate expansions. This control object links query understanding to retrieval execution. It contains the active actor, topic, issue, and document filters, alongside explicit and inferred article numbers, ensuring that subsequent retrieval layers are guided by a unified, structured intent.

Forced reference promotion. When a user query matches high-precision legal rules, the routing engine identifies specific statutory provisions as mandatory contexts

that must be retrieved, regardless of their vector similarity score. These are represented as high-confidence forced references inside the set $\mathcal{R}_{\text{forced}}$. To prioritize these legally required documents, the system fetches them directly by their coordinates and promotes them. The system uses a deterministic score adjustment for each promoted forced-reference chunk d_{forced} using the following formula:

$$\text{Score}(d_{\text{forced}}) = \max_{h \in S_k} \text{Score}(h) + \Delta_{\text{forced}} - \rho \cdot 10^{-4}$$

where S_k is the set of seed chunks returned by the hybrid vector search, $\max_{h \in S_k} \text{Score}(h)$ is the maximum relevance score among those baseline seeds, Δ_{forced} is the forced-reference score margin (configured as a positive constant), and ρ represents the ordinal rank of the forced record. This score adjustment ensures that every mandatory forced-reference chunk receives a score higher than the highest-scoring vector search hit, helping prioritize high-confidence legal coordinates.

Coordinate fallback. Beyond explicit forced references, the system adds a fallback path to retrieve specific statutory articles that the baseline hybrid search might omit due to vector space limitations. The system compares the required article coordinates in the query intent (A_{required}) against the article numbers actually present in the retrieved hybrid search hits ($A_{\text{retrieved}}$). The set of missing target articles, A_{missing}, is computed via:

$$A_{\text{missing}} = A_{\text{required}} \setminus A_{\text{retrieved}}.$$

If A_{missing} is non-empty, the system fetches the missing statutory chunks directly from the record store. If the fallback is triggered by an explicit mention of an article number in the user's query (explicit fallback), the fetched chunks are promoted slightly above the current hits, receiving a score boost of $+10^{-3}$ relative to the maximum existing score. If the fallback is triggered by rules that infer article numbers from topic/issue mappings (inferred fallback), the chunks receive a minor score penalty of -10^{-3} relative to the maximum score. The explicit fallback receives a stronger score adjustment, while the inferred fallback is kept more conservative. This fallback mechanism reduces the risk of missing explicit legal coordinates.

Adaptive context window. To balance retrieval coverage and prompt noise, the system implements an adaptive context windowing policy. When a query involves highly complex legal issues or comparative intent, a narrow context window may truncate critical legislative provisions, whereas a broad window for simple lookups introduces unnecessary noise that degrades generation quality. To resolve this, the system dynamically scales the primary article retrieval limit, L_{primary}, using the following adaptive relation:

$$L_{\text{primary}} = \begin{cases} 8, & \text{if } T_q \cap \Theta_{\text{complex}} \neq \emptyset \text{ or } I_q \cap \Phi_{\text{complex}} \neq \emptyset, \\ 4, & \text{otherwise.} \end{cases}$$

where T_q is the set of query types extracted in the intent, I_q is the set of active issue filters, and Θ_{complex} and Φ_{complex} represent the configured sets of complex query types and complex issues. For simple queries, the limit defaults to 4 articles, while secondary article limits are kept smaller (typically 2 to 4). This adaptive limit controls prompt noise while ensuring sufficient coverage for complex queries.

Final context assembly. Because the seed chunks S_k from the hybrid retrieval layer share identical coordinates with graph nodes, they are directly mapped into Neo4j graph nodes for traversal. The system then traverses the knowledge graph to gather

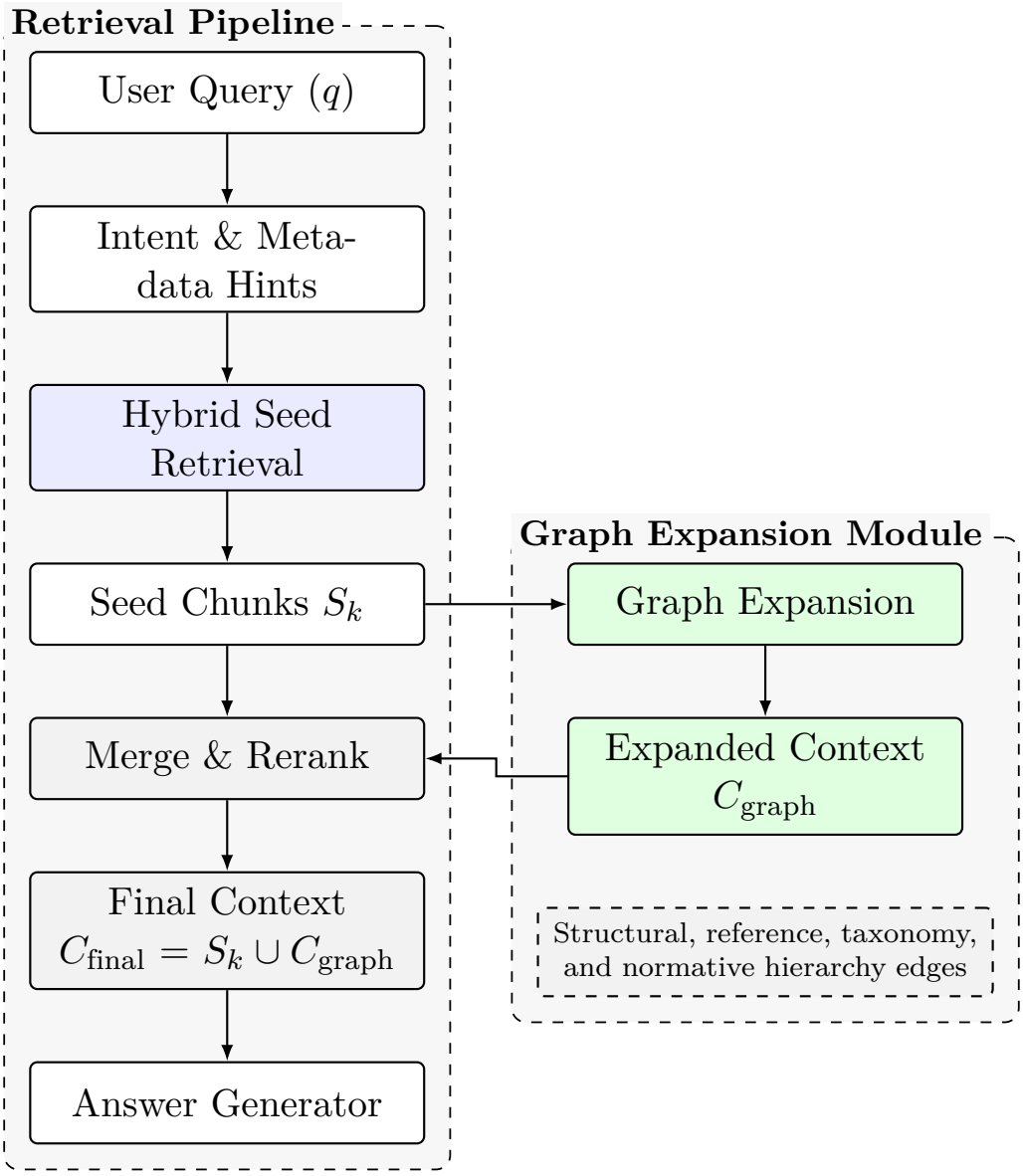

Figure 3.4: Graph-augmented retrieval flow combining hybrid seed retrieval and legal graph expansion.

topologically associated legal provisions—such as parent-child structural units, cross-document circular details, or normative hierarchy links. The final retrieved context set C_{final} is consolidated by merging the baseline seed chunks S_k, the graph-expanded context C_{graph}, and the coordinate fallback chunks C_{fallback}:

$$C_{\text{final}} = \text{Rerank}\left(S_k \cup C_{\text{graph}} \cup C_{\text{fallback}}\right).$$

This final context set is passed to the grounded answer generator, where evidence selection, reranking, and citation validation are executed, as described next in Section 3.7.

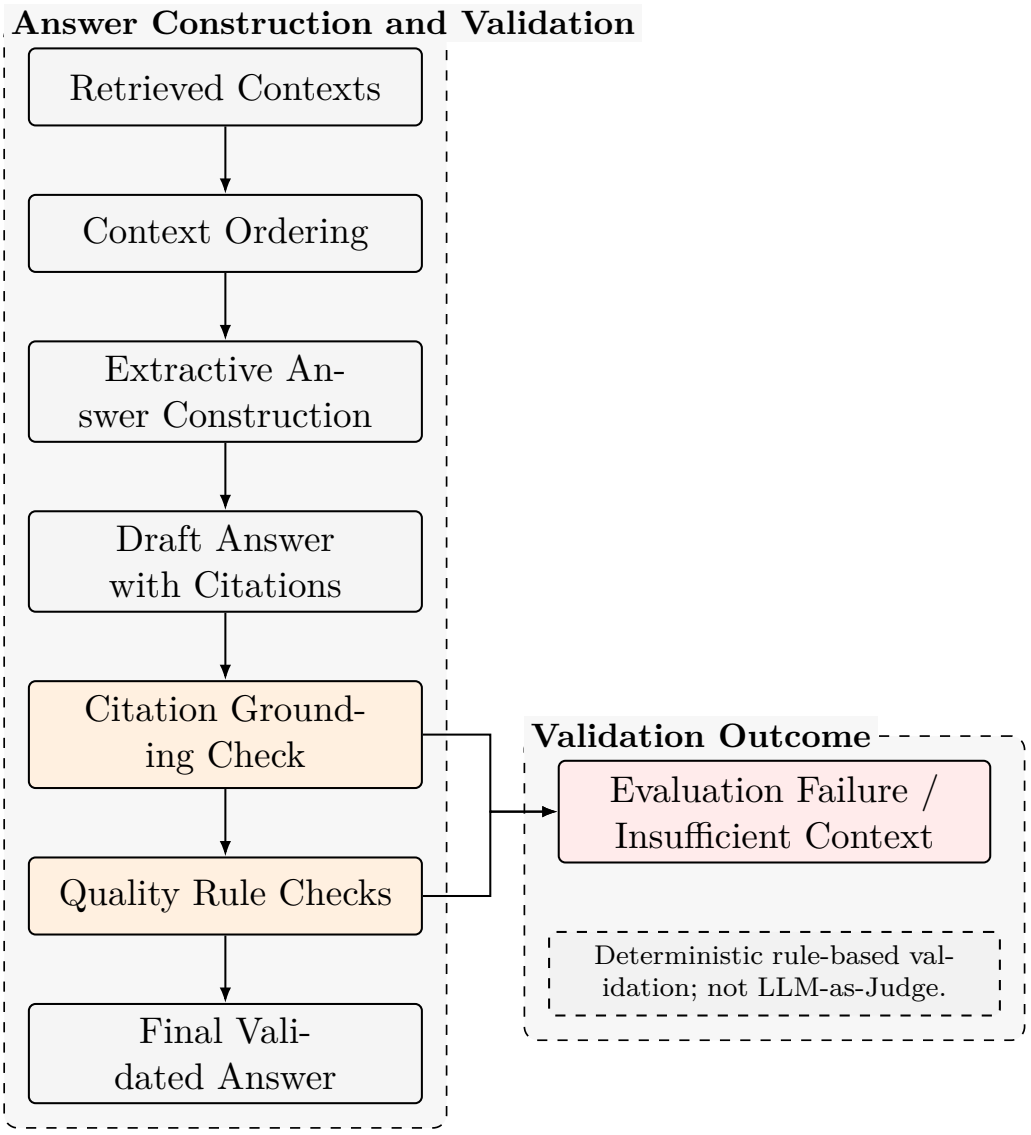

Figure 3.5: Answer generation and deterministic citation validation flow.

3.7 Grounded Answer Generation with Legal Citations

The final stage of the pipeline converts the consolidated, reranked retrieval contexts into a citation-grounded answer (Figure 3.5). Directly presenting raw language model outputs in professional legal settings can introduce risks such as semantic hallucinations and citation misattributions. To reduce the risk of unsupported claims, the answering layer runs retrieved contexts through semantic reranking, normative-rank context budgeting, and a deterministic validation gate. The system supports structured answer generation, but final quality evaluation relies on deterministic rule-based validation. Notably, answer quality is evaluated using deterministic rule-based validation, not human legal expert review or LLM-as-Judge evaluation.

Semantic reranking. To improve context relevance, the pooled context candidates are processed by a semantic cross-encoder model. While standard vector search is effective for broad initial retrieval, a cross-attention transformer provides finer granularity by

analyzing attention weights across the query-context pairs. Candidate chunks within a parameterized limit ($N = 24$) are evaluated against the query q using a cross-encoder model. To preserve the initial search hierarchy, the system does not discard the initial search order; instead, it computes a Reciprocal Rank Fusion (RRF) score by combining the chunk's heuristic retrieval rank $R_{\text{heuristic}}(d)$ with its semantic cross-encoder rank $R_{\text{semantic}}(d)$:

$$\text{RRF}_{\text{fused}}(d) = \frac{1}{K_{\text{RRF}} + R_{\text{heuristic}}(d)} + \frac{1}{K_{\text{RRF}} + R_{\text{semantic}}(d)}$$

where the rank-stabilization constant K_{RRF} is set to 60. Chunks are sorted in descending order of their RRF scores to prioritize the most semantically relevant evidence.

Normative-rank context budgeting. Following semantic reranking, the system executes a context budgeting algorithm to select the evidence window. Statutory provisions in Vietnamese labor law carry different levels of authority: Rank 1 represents primary Codes, Rank 2 denotes administrative Decrees, and Rank 3 corresponds to ministerial Circulars. To prevent detailed implementing guidelines from overshadowing primary statutory provisions, the system enforces a fixed budget limit $B(r)$ based on document rank r:

$$B(r) = \begin{cases} 3, & r = 1 \text{ (Codes/Laws)}, \\ 2, & r = 2 \text{ (Decrees)}, \\ 2, & r = 3 \text{ (Circulars)}. \end{cases}$$

Additionally, contexts are dynamically deduplicated using a unique article key composed of the document identifier and article number. This prevents multiple chunks from a single article from consuming the budget, encouraging document diversity in the final prompt context.

Citation-aware answer construction. Grounded answers are generated from the budgeted contexts using structured prompt formats. The answering layer can force a JSON schema with fields for the natural language answer, legal basis, evidence quotes, insufficient-context flag, and notes. The exact API field names are shown in the implementation example in Chapter 4 and in Appendix G. To maintain a consistent structure, the answer starts with a lead sentence generated by a semantic anchoring transition (Γ), which maps user query patterns to direct legal transitions:

$$\text{Lead}_{\text{answer}} = \Gamma(q, c_1),$$

where q is the query and c_1 is the highest-ranked context chunk. This structuring guides the output to flow logically from primary laws to secondary implementing regulations, using only retrieved contexts and the designated legal basis while avoiding unsupported article numbers.

Deterministic validation gate. Before the generated answer is returned, a validation gate evaluates the output to check whether citations are supported. The validation check computes the logical conjunction of five rule-based guardrails:

$$\Phi_{\text{validation}} = \phi_{\text{citation}} \wedge \phi_{\text{retrieved}} \wedge \phi_{\text{article}} \wedge \phi_{\text{rank}} \wedge \phi_{\text{uncertainty}}.$$

These individual validation guardrails check whether citations are supported within the retrieved corpus:

- **Inline Citation Guard (ϕ_{citation}):** Confirms that the answer text contains inline citations pointing to the legal basis, unless the system has flagged that context is insufficient:

$$\phi_{\text{citation}} = \mathbb{I}\left(\text{Answer contains citation}\right) \vee \mathbb{I}\left(\texttt{insufficient\_context} = \text{true}\right)$$

- **Retrieval Boundary Guard ($\phi_{\text{retrieved}}$):** Checks whether all citations listed in the legal basis are present within the retrieved context space ($\mathcal{C}_{\text{retrieved}}$), blocking any out-of-bounds references:

$$\phi_{\text{retrieved}} = \mathbb{I}\left(\text{Citations} \in \mathcal{C}_{\text{retrieved}}\right)$$

- **Article Numeral Verification (ϕ_{article}):** Verifies that any article numbers mentioned in the answer text (A_{answer}) or the legal basis (A_{basis}) are a subset within the articles of the retrieved contexts (A_{contexts}):

$$A_{\text{answer}} \cup A_{\text{basis}} \subseteq A_{\text{contexts}}$$

- **Normative Rank Priority Gate (ϕ_{rank}):** Checks that the answer does not cite low-rank circulars while ignoring higher-authority codes present in the retrieved contexts. It requires $r^{\min}_{\text{cited}} \leq r^{\min}_{\text{available}}$, where $r^{\min}$ denotes the highest authority level (minimum numeric value):

$$\phi_{\text{rank}} = \mathbb{I}\left(r^{\min}_{\text{cited}} \leq r^{\min}_{\text{available}}\right)$$

- **Uncertainty Disclosure Gate ($\phi_{\text{uncertainty}}$):** Confirms that if the context is insufficient, the text includes standard uncertainty indicators to signal limits in evidence coverage:

$$\phi_{\text{uncertainty}} = \mathbb{I}\left(\text{Answer contains uncertainty}\right) \vee \mathbb{I}\left(\texttt{insufficient\_context} = \text{false}\right)$$

If any guardrail fails ($\Phi_{\text{validation}} = 0$), the stochastically generated answer is flagged as invalid, and the system uses fallback mechanisms.

Extractive fallback. When the structured generator fails to produce a valid response format or flags invalid outputs via the validation gate, the system triggers a deterministic extractive fallback. This fallback mechanism bypasses language model generation entirely. By leveraging the structural metadata from the preprocessing stage, the pipeline extracts canonical paragraph snippets directly from the retrieved context chunks. It filters out low-information lines using a minimum token threshold and concatenates the remaining high-relevance snippets, anchoring each statement to its native citation. This approach ensures that when text generation fails, the system still provides a reliable, citation-supported answer constructed entirely from the retrieved source text.

Interpretation and limitations. The rule-based validation gate and extractive fallback are designed to improve reproducibility and reduce hallucinated citations within the retrieved corpus. However, this benchmark-specific validation is deterministic and rule-based; it does not replace professional legal expert review, and it does not establish legal correctness beyond this benchmark. Instead, it serves as a software-level verification step to check whether the output aligns with the retrieved documents.

Chapter 4

Implementation

4.1 Technology Stack

The implemented legal question-answering system is constructed as a modular Python-based pipeline for reproducible retrieval and citation grounding. The backend orchestration layer is written in Python (version 3.10+) and exposes high-level service interfaces via FastAPI. Data persistence, semantic indexing, and relational traversal are partitioned across two specialized database layers. For vector-based and keyword-based search, the system integrates a Qdrant vector database collection, which maintains dense high-dimensional semantic encodings and sparse index structures. For relational cross-reference modeling and topological query expansion, the system utilizes a Neo4j graph database instance configured with the APOC (Awesome Procedures on Cypher) plugin library.

Table 4.1: Technology stack.

Layer	Technology	Purpose
Backend	Python, FastAPI	API and orchestration
Embeddings	Sentence Transformers	Vietnamese sentence embeddings
Vector database	Qdrant	Dense and sparse retrieval
Graph database	Neo4j	Legal hierarchy and references
Evaluation	Python scripts	Benchmark reporting

The dense embeddings are generated locally using the Sentence Transformers framework, employing the pre-trained `keepitreal/vietnamese-sbert` model (yielding 384-dimensional vectors). To ensure high reproducibility, the Neo4j Community Edition 5.26 service is containerized using Docker and managed programmatically via Docker Compose. The compose file is `docker-compose.neo4j.yml`. Runtime connection values are parameterized through environment variables, including `QDRANT_URL`, `NEO4J_URI`, and `NEO4J_AUTH`. Active model and graph traversal settings are also read from the central `.env` file.

Table 4.2: Final implementation configuration.

Component	Final setting
Embedding model	`keepitreal/vietnamese-sbert`
Vector store	Qdrant
Graph database	Neo4j
Indexed corpus size	1,556 chunks
Graph expansion	Enabled
Scope guard	Enabled
Procedural routing	Disabled by default

4.2 Data Processing Implementation

The transformation of raw legal texts into validated, queryable vector and graph structures follows a sequential Python pipeline located in the `scripts/` directory. The process begins with clean text validation using `validate_curated_legal_texts.py`. Validation reports are written under `artifacts/validation/`. The chunk generation step then uses `build_legal_chunks.py` to parse curated legal text files and write chunk artifacts. Next, `enrich_legal_chunks.py` maps semantic attributes to each chunk, including topics, actors, issue types, document types, and legislative normative ranks. The final processing step extracts relative and document-level legal cross-references using `build_reference_edges.py`. Unresolved citations are flagged through `analyze_unresolved_reference_edges.py` for diagnostic review.

4.3 Vector Index and Hybrid Retrieval Implementation

The vector indexing layer ingests the 1,556 enriched legal chunks and populates the Qdrant vector store. This process is orchestrated by running `build_index.py` with the configured Qdrant collection name. During indexing, each chunk's textual content is converted into a 384-dimensional dense vector using the configured Vietnamese SBERT embedding model.

The resulting vector points are uploaded to Qdrant alongside structured payload metadata. This payload stores the chunk identity, primary document, article number, clause reference, normative rank, topic labels, and original Vietnamese citation surface. Hybrid retrieval fuses dense vector similarity searches with sparse token matching, combining semantic proximity with exact matching for article numbers, statutory coordinates, and keywords. The payload fields are used during retrieval to enforce runtime filters, support downstream citation validation, and enable deterministic context reranking before prompting.

4.4 Neo4j Graph Implementation

To construct the relational backbone of the corpus, the system deploys a local Neo4j 5.26 Community Edition container via the `docker-compose.neo4j.yml` compose file. The

legal graph is loaded and validated using `build_legal_graph.py`. The script connects through the Bolt protocol on port 7687 using credentials configured in the `.env` file.

The graph model defines node types representing the legislative hierarchy, including documents, articles, clauses, points, appendices, and evidence chunks, as well as semantic nodes for topics, actors, and issue types. Relationships are loaded from the processed reference-edge artifacts, including structural hierarchy links and taxonomy edges. Legal cross-references are represented using reference, detail, and guidance edges. Unique constraints and indexes are applied to node identifiers to ensure upsert integrity. In this build, 111 unresolved references were flagged and excluded from the Neo4j instance to prevent traversing dead-end or invalid citation paths.

4.5 Graph-Augmented Retriever Implementation

The graph-augmented retriever is implemented in the backend retrieval module. Its execution flow coordinates Qdrant searches and Neo4j graph queries to assemble a candidate legal context window. The process starts with seed retrieval: the query is routed to Qdrant, where hybrid retrieval fetches the top-k seed chunks, typically 10. During node mapping, the retriever extracts seed chunk identifiers and maps them to the matching evidence nodes in Neo4j. Topological expansion then begins from the mapped seed nodes. The retriever executes a parameterized Cypher query through the Bolt driver to traverse reference edges, taxonomy links, and structural parents, with the expansion depth controlled by `LEGAL_GRAPH_EXPANSION_DEPTH`. In the context assembly stage, the retriever merges the graph-expanded chunks with the original seeds, resolves score weights based on topological distance, deduplicates records by chunk ID, and selects the top-ranked contexts for downstream generation.

4.6 Runtime QA Pipeline

At runtime, the system combines retrieval, graph expansion, scope checking, answer construction, and validation. Algorithm 4.1 summarizes the final pipeline in implementation-level pseudo-code.

```
Input: user_query
Output: answer_payload

intent = detect_intent(user_query)
seed_chunks = qdrant.hybrid_search(user_query, top_k=10)

if graph_expansion_enabled:
   graph_chunks = neo4j.expand_from(seed_chunks,
                                    edge_types=[DETAILS, GUIDED_BY, REFERENCES],
                                    depth=LEGAL_GRAPH_EXPANSION_DEPTH)
else:
   graph_chunks = []

contexts = merge_deduplicate_and_rerank(seed_chunks, graph_chunks)
contexts = apply_context_budget(contexts, max_answer_contexts)

if scope_guard_enabled and is_out_of_scope(user_query, contexts):
```

```
   return insufficient_context_payload(user_query)

draft = generate_or_extract_answer(user_query, contexts)
validation_status = validate_citations(draft, contexts)

if validation_status fails:
   draft = build_extractive_fallback(user_query, contexts)
   validation_status = validate_citations(draft, contexts)

return format_answer_payload(draft, validation_status)
```

Listing 4.1: Runtime QA pipeline pseudo-code.

4.7 Grounded Answer Generator and Validation

The grounded answer generation layer is implemented in the answering modules of the backend package. When LLM provider access is enabled through environment variables, such as `LLM_PROVIDER=groq`, the system constructs a system prompt containing the assembled context block and the configured answer JSON schema. The LLM is instructed to output a parsed JSON payload containing the natural language answer, specific parenthetical citation labels, and exact evidence quotes from the retrieved text.

Before answer synthesis, the final candidate applies deterministic intent gating and a scope guard. The scope guard checks whether the query belongs to a known unsupported topic family and whether the retrieved evidence is sufficient for that topic. If the query is unsupported and the evidence is insufficient, the pipeline returns a structured insufficient-context output with an empty legal-basis field. This prevents unrelated in-corpus citations from being used to answer questions outside the indexed scope. The guard is rule-based and limited to the unsupported topic families defined for this benchmark.

To ensure safety and prevent hallucinations, the generation layer runs the response through a deterministic validation pipeline. The validation checks enforce three main rules: (1) every generated citation must map exactly to an article present in the retrieved context; (2) every statement of legal basis must match the original Vietnamese wording; and (3) no unsupported article numbers are allowed. If the validation check fails, or if the LLM provider is disabled, the system automatically triggers an extractive fallback using `build_extractive_answer_payload`. This fallback directly extracts the validated legal provisions from the retrieved chunks and formats them as the final output. This fallback is used to reduce unsupported generation and ensure alignment with the indexed corpus.

The runtime API returns a structured payload so that the answer text and its legal bases can be checked separately. The following compact example shows the intended shape of the output; the wording is shortened for readability.

```
{
  "query": "Nguoi lao dong duoc dinh nghia nhu the nao?",
  "answer": "Nguoi lao dong la nguoi lam viec cho nguoi su dung lao dong theo thoa
  ↪  thuan, duoc tra luong va chiu su quan ly, dieu hanh, giam sat cua nguoi su
  ↪  dung lao dong.",
  "legal_basis": [
    "Bo luat so 45/2019/QH14, Dieu 3, khoan 1"
  ],
```

```
  "validation_status": {
    "citation_valid": true,
    "unsupported_articles": [],
    "insufficient_context": false
  }
}
```


4.8 Evaluation Scripts and Reproducibility

The evaluation pipeline contains scripts designed to measure retrieval performance and end-to-end QA safety over the final 100-query benchmark. The evaluations are fully reproducible using the commands listed in Appendix F. The retrieval ablation compares hybrid-only and graph-augmented retrieval on the 94 in-corpus benchmark queries. The end-to-end evaluation runs the full retrieval-to-generation pipeline with the final candidate configuration and validation checks. The adjusted benchmark split metrics are then computed from the saved result files. Overall execution and diagnostic checks are managed by the benchmark coordinator.

Reproducibility is supported by keeping the final candidate settings explicit and by computing benchmark metrics from saved machine-readable result files. The evaluation scripts rebuild retrieval outputs, run the end-to-end pipeline, and recompute the adjusted split metrics using fixed benchmark annotations. Appendix F lists the PowerShell commands used for these steps, so the reported metrics can be regenerated without relying on manual judgment or an LLM-as-Judge step.

4.9 Example Use Cases

The utility of the implementation is demonstrated across several common legal inquiry patterns evaluated on the diagnostic benchmark. For minor worker employment, a query about hiring a 14-year-old worker triggers seed retrieval of Labor Code Article 143, followed by graph-augmented traversal to Circular 09/2020/TT-BLDTBXH to extract the administrative list of allowed occupations. For retirement age table lookup, an inquiry about female retirement ages in 2026 is routed to Labor Code Article 169, which is linked through a detail edge to Appendix I of Decree 135/2020/ND-CP, allowing the retriever to capture the relevant table row and output 57 years. For labor contract content, a question about mandatory contract details retrieves Article 21 and then traverses the graph to Circular 10/2020/TT-BLDTBXH Article 3 for administrative guidance. For dismissal dispute procedure, a multi-hop query about dismissal mediation connects Labor Code Article 188 to Article 32 of the Civil Procedure Code (BLTTDS 2015), linking substantive rights and court jurisdiction.

Chapter 5

Evaluation and Results

5.1 Evaluation Setup and Benchmark Design

This chapter evaluates the Vietnamese labor-law RAG system on the final 100-query benchmark. The benchmark is used as the only final evaluation benchmark in this thesis. It contains 94 in-corpus queries and 6 out-of-corpus refusal tests.

The evaluation is bounded by the indexed corpus. The corpus contains six Vietnamese labor-law documents, including the Labor Code, selected decrees, selected circulars, and a labor-related subset of the Civil Procedure Code. These documents were divided into 1,556 legal chunks. The retrieval layer stores these chunks in Qdrant and uses Neo4j for graph-based expansion over legal structure and cross-references.

The final candidate evaluated in this chapter uses graph-augmented retrieval together with deterministic intent gating and the scope guard described in Chapter 4.

Table 5.1: Corpus and index statistics.

Item	Value
Corpus documents	6
Legal chunks	1,556
Vector index count	1,556
Neo4j documents	6
Neo4j articles	411
Neo4j clauses	1,235
Neo4j points	774
Neo4j appendices	35
Neo4j evidence chunks	1,556
Loaded `REFERENCES` edges	290
Loaded `DETAILS` reference edges	329
Loaded `GUIDED_BY` reference edges	329
Loaded taxonomy metadata edges	6,099
Loaded normative hierarchy edges	20
Unresolved references excluded from graph traversal	111

The benchmark covers several query types. It includes direct QA, definition QA, document guidance QA, exception-based QA, multi-hop QA, table and appendix lookup,

hard-negative citation QA, paraphrased real-user QA, labor dispute and procedure QA, and out-of-corpus QA. The last group is designed to test whether the system refuses questions that are not supported by the indexed corpus.

Table 5.2: Final benchmark composition.

Category	Queries
Direct QA	19
Definition QA	7
Document guidance QA	10
Exception-based QA	11
Multi-hop QA	7
Table and appendix lookup	10
Hard-negative citation QA	5
Paraphrased real-user QA	5
Labor dispute and procedure QA	5
Procedure QA	7
Comparison QA	4
Scenario-based QA	4
Out-of-corpus QA	6
Total	100

Each in-corpus query has a manually annotated set of required citations. Some queries also include forbidden citations. These forbidden citations represent legal provisions that are similar to the question but should not be used as the main basis for the answer. The 6 out-of-corpus queries have empty required citation sets because the correct behavior is refusal or an insufficient-context answer.

The evaluation uses deterministic checks. It does not use human legal expert review. It also does not use an LLM-as-Judge. This makes the result reproducible, but it also means that the evaluation checks structural correctness rather than full legal reasoning quality.

5.2 Retrieval Evaluation Methodology

The retrieval evaluation measures whether the system finds the legal evidence needed to answer a query. In legal QA, semantic similarity is not enough. A passage can be similar to the question but still cite the wrong article, the wrong document, or an incomplete rule.

Let Q be the full benchmark. The benchmark is split into two sets:

$$Q = Q_{\text{in}} \cup Q_{\text{ooc}},$$

where Q_{in} is the set of in-corpus queries and Q_{ooc} is the set of out-of-corpus queries. In this benchmark:

$$|Q| = 100, \quad |Q_{\text{in}}| = 94, \quad |Q_{\text{ooc}}| = 6.$$

Retrieval metrics are computed only on Q_{in}. The out-of-corpus queries have no required citations. Including them in retrieval coverage would create artificial scores because there is no gold citation set to retrieve.

For an in-corpus query $q \in Q_{\text{in}}$, let $G(q)$ be the required citation set. Let $R_{10}(q)$ be the top-10 retrieved contexts. Let $\mathcal{C}_{10}(q)$ be the set of citation coordinates found in those contexts:

$$\mathcal{C}_{10}(q) = \{C(d) \mid d \in R_{10}(q)\}.$$

The per-query recall is:

$$\text{Recall}\,@10(q) = \frac{|G(q) \cap \mathcal{C}_{10}(q)|}{|G(q)|}.$$

The reported Recall@10 is the macro-average over the 94 in-corpus queries:

$$\text{Recall}\,@10 = \frac{1}{|Q_{\text{in}}|} \sum_{q \in Q_{\text{in}}} \text{Recall}\,@10(q).$$

Required Citation Coverage is also computed over Q_{in}. It reports the average fraction of required citations recovered in the top-10 context window. A separate retrieval pass rate is used to measure stricter query-level success. A query passes retrieval only when all required citations are retrieved and no forbidden citation rule is violated.

Forbidden Citation Violation Rate measures how often a retrieved context includes a benchmark-defined forbidden citation:

$$\text{ForbiddenViolationRate} = \frac{1}{|Q_{\text{in}}|} \sum_{q \in Q_{\text{in}}} \mathbb{I}\left[F(q) \cap \mathcal{C}_{10}(q) \neq \emptyset\right],$$

where $F(q)$ is the forbidden citation set for query q.

5.3 Answer and Citation Evaluation Methodology

The end-to-end evaluation checks the full pipeline. It includes retrieval, answer generation, citation grounding, and answer quality validation.

For in-corpus queries, the system must retrieve the required legal evidence, generate an answer, cite only retrieved sources, and pass the quality checks. The quality checks look for a direct answer, the required legal rule, appropriate numeric or yes/no content when needed, and the absence of low-information quotes.

Citation grounding is checked by comparing the citations used in the answer with the citations available in the retrieved contexts. Let B_q be the legal basis citations used in the answer. Let $\mathcal{C}(C_q)$ be the citation set available in the contexts passed to the generator. The citation check requires:

$$B_q \subseteq \mathcal{C}(C_q).$$

The validator also checks that article numbers mentioned in the answer are present in the retrieved contexts. This reduces citation hallucination and unsupported article references.

Out-of-corpus queries use a different pass rule. They pass only if the final answer refuses the question or reports insufficient indexed context. They fail if the system answers by using unrelated in-corpus citations. They also fail if the system gives specific legal values that are not present in the indexed corpus.

The final end-to-end score is therefore an adjusted score. For $q \in Q_{\text{in}}$, the normal retrieval, answer, citation, and quality checks are used. For $q \in Q_{\text{ooc}}$, the refusal or insufficient-context check is used. This adjusted score is the final end-to-end result reported in this chapter.

5.4 Retrieval Results

Table 5.3 reports graph-augmented retrieval performance on the 94 in-corpus queries. Recall@10 and Required Citation Coverage are not computed over the 6 out-of-corpus queries.

Table 5.3: Retrieval results on the final benchmark.

Metric	Value
In-corpus queries	94
Recall@10	0.835
Required Citation Coverage	0.835
Forbidden Citation Violation Rate	0.043
In-corpus retrieval pass rate	0.766

The result shows that retrieval is still the main bottleneck. The system recovers most required citations, but it does not consistently retrieve the full evidence set for harder queries. This is visible in the gap between Required Citation Coverage of 0.835 and the stricter retrieval pass rate of 0.766.

Table 5.4 compares hybrid-only retrieval with graph-augmented retrieval on the same 94 in-corpus queries. Graph expansion improves Recall@10 and Required Citation Coverage from 0.748 to 0.835. It also improves the retrieval pass rate from 0.660 to 0.766. The Forbidden Citation Violation Rate increases from 0.032 to 0.043, which shows that graph expansion can add useful evidence but can also introduce some distracting legal bases.

Table 5.4: Retrieval ablation on the 94 in-corpus queries.

Mode	Queries	Recall@10	Required Citation Coverage	Forbidden Citation Violation Rate	Retrieval Pass Rate
Hybrid-only	94	0.748	0.748	0.032	0.660
Graph-augmented	94	0.835	0.835	0.043	0.766

The failures are concentrated in paraphrased user queries, multi-hop queries, table and appendix lookup, hard-negative citation queries, and labor-dispute procedure queries. These groups require more than lexical matching. They often require the retriever to connect informal wording to a legal concept, follow cross-document references, or find a specific appendix row.

The Forbidden Citation Violation Rate is 0.043. This means that some hard-negative cases still retrieve a similar but wrong legal basis. These cases are important because they can lead the answer generator toward a legally distracting provision even when the citation itself is present in the corpus.

5.5 End-to-End Results

Table 5.5 reports the adjusted end-to-end results for the final 100-query benchmark. The adjusted score uses normal in-corpus validation for the 94 in-corpus queries and refusal validation for the 6 out-of-corpus queries.

Table 5.5: Adjusted end-to-end results on the final benchmark.

Metric	Value
Total benchmark queries	100
In-corpus queries	94
Out-of-corpus refusal tests	6
Adjusted end-to-end pass rate	0.780
Retrieval pass rate	0.780
Answer pass rate	0.970
Citation validation pass rate	1.000
Quality validation pass rate	0.970
Average final quality score	93.9693
Low-information quotes	2
Unsupported article numbers	0
Unretrieved citations	0
Out-of-corpus QA pass rate	1.000

The adjusted end-to-end pass rate is 0.780. This result is lower than the answer and citation validation rates because many failures still occur at retrieval time. If required evidence is missing from the retrieved context window, the final answer cannot pass the full benchmark rule even if it is well formed and grounded in the retrieved contexts.

The citation validation pass rate remains 1.000. This means the system did not cite sources outside the retrieved context in the final evaluation. The result indicates that the citation guard is effective at preventing unsupported citation surfaces. However, this does not solve missing retrieval. A grounded answer can still be incomplete or based on the wrong retrieved context if the retriever fails to find the required evidence.

The out-of-corpus QA pass rate is 1.000. In the final candidate, the scope guard and intent gate caused all 6 out-of-corpus tests to return refusal or insufficient-context behavior. This improves bounded-corpus safety in the benchmark, but the guard is rule-based and limited to known unsupported topic families.

5.6 Case Studies

This section gives three examples from the final benchmark. They are used to explain the quantitative results, not to add new metrics.

Successful in-corpus case. The query asks: "Nu nghi huu nam 2026 thi bao nhieu tuoi va can cu van ban nao?" The required evidence includes the Labor Code rule on retirement age, Decree 135/2020/ND-CP Article 4, and the retirement age table in Appendix I. The system retrieved the required retirement-age table and returned the correct answer: female workers retiring under normal conditions in 2026 are at retirement age 57. This case passed retrieval, answer, citation, and quality checks.

Successful out-of-corpus refusal case. The query asks: "Nam 2026 luong toi thieu vung I, II, III, IV la bao nhieu?" The indexed corpus does not contain the legal instrument needed to answer this question. In the final candidate, the scope guard and intent gate detected that the query was outside the supported benchmark scope and that the retrieved evidence was insufficient. The system returned an insufficient-context response with an empty `legal_basis` field. This case passed the out-of-corpus QA rule.

Remaining retrieval failure case. The query asks: "Cong ty noi hang it viec nen chuyen toi sang viec khac gan 3 thang, khong hoi y kien toi, vay co on khong?" The required evidence is Labor Code Article 29 on temporary transfer to another job. The retriever did not retrieve Article 29. Instead, the top results included unrelated wage and general employment provisions. The answer was grounded in retrieved text, but the retrieved text did not contain the required rule. This case failed with `retrieval_missing_required_context`.

5.7 Error Analysis and Limitations

The main error pattern is `retrieval_missing_required_context`. The benchmark indicates that the system fails when the required legal basis is not retrieved in the top-10 context window. This affects paraphrased user queries, multi-hop retrieval, table and appendix lookup, hard-negative citation cases, and labor-dispute procedure queries.

Hard-negative over-expansion is another error source. Four in-corpus cases retrieved forbidden citations. These cases show that the retriever can still promote a similar but incorrect legal basis. This is risky in labor-law QA because related provisions may use similar wording but answer a different legal question.

Paraphrased user queries are difficult because the user does not name the legal article. The retriever must infer the legal issue from informal wording. In the final benchmark, these cases remain a source of end-to-end failure. This suggests that query understanding and query rewriting need more work.

Table and appendix lookup remains limited. Four table or appendix lookup items failed. These failures occur because the needed evidence is often a specific appendix chunk or table row. Numeric and table-heavy provisions are harder to retrieve than ordinary article text.

Labor-dispute procedure questions also show weakness. These queries require the system to connect substantive labor rules with civil procedure provisions. The result shows that cross-code retrieval remains difficult for the current pipeline.

The out-of-corpus tests passed in the final candidate. This result is useful, but it should not be read as general out-of-scope detection. The current scope guard is rule-based and limited to known unsupported topic families.

The evaluation has one further limitation. It does not include human legal expert review. The deterministic checks are useful for reproducible software evaluation, but they cannot decide whether every legal interpretation is correct in practice. Future work should include expert review and a broader corpus before using the system in high-stakes settings.

Overall, the final 100-query benchmark shows a mixed result. Citation grounding and out-of-corpus refusal are strong in this benchmark, but in-corpus retrieval robustness remains limited.

Chapter 6

Discussion

6.1 Key Findings

The final evaluation shows a mixed result. The system performs well on many structured and direct in-corpus questions. It can retrieve and cite legal provisions from the indexed corpus, and the validation layer keeps final citations tied to retrieved context.

The strongest result is citation grounding. On the final 100-query benchmark, the citation validation pass rate is 1.000. This means that the final answers did not cite legal bases outside the retrieved context. The answer pass rate is 0.970 and the quality validation pass rate is 0.970, which also indicates that most generated answers are well formed when the needed evidence is available.

The main weakness is in-corpus retrieval. On the 94 in-corpus queries, graph-augmented retrieval achieves Recall@10 of 0.835 and Required Citation Coverage of 0.835. The Forbidden Citation Violation Rate is 0.043. The adjusted end-to-end pass rate over all 100 queries is 0.780. This gap shows that missing or distracting retrieved evidence still affects the full pipeline even when citation validation remains strong.

The 6 out-of-corpus queries are also important. The out-of-corpus QA pass rate is 1.000 in the final benchmark. The scope guard and intent gate prevented unsupported answers for these tests by returning refusal or insufficient-context behavior.

6.2 Impact of Hierarchical Chunking

Hierarchical chunking remains useful for legal retrieval. Vietnamese legal documents are organized by document, article, clause, point, and appendix. If these structures are split into arbitrary text windows, the retriever may lose the legal context needed for citation and interpretation.

The implemented chunking pipeline preserves legal coordinates and parent-child relations. This helps the system return evidence with clear citation paths. It also supports deterministic validation, because each generated citation can be checked against retrieved metadata.

However, hierarchical chunking is not enough by itself. The final benchmark shows that some questions still fail because the required chunk is not retrieved. This happens especially when a user uses informal wording, when the answer depends on a table or appendix row, or when the query requires procedural information from another legal document.

6.3 Graph-Augmented Retrieval

Graph expansion helps connect retrieved seed chunks to related legal provisions. This is useful when a Labor Code article is implemented by a decree, when a circular gives detailed guidance, or when a civil procedure provision is linked to a substantive labor-law issue. These relations are difficult to recover using only semantic similarity.

The retrieval ablation shows that graph-augmented retrieval improves over hybrid-only retrieval. Recall@10 increases from 0.748 to 0.835, Required Citation Coverage increases from 0.748 to 0.835, and the retrieval pass rate increases from 0.660 to 0.766.

One unexpected result is that graph expansion improves retrieval recall but also introduces a small increase in forbidden citation violations. The Forbidden Citation Violation Rate increases from 0.032 for hybrid-only retrieval to 0.043 for graph-augmented retrieval. This suggests that structural proximity in the graph is useful, but it is not always the same as legal relevance. A nearby provision can be related to the same legal topic while still being the wrong basis for a specific answer.

If I were to redesign the retrieval pipeline, I would separate query understanding from graph expansion more clearly. Graph traversal can add useful related provisions, but it cannot fix weak seed retrieval for paraphrased user questions. When the initial seed misses the legal issue, graph expansion may follow the wrong neighborhood and add more context without adding the required rule.

6.4 Role of Normative Hierarchy in Legal QA

Vietnamese legislation has a vertical authority structure. Codes and laws establish broad rules, while decrees and circulars provide implementation details. The graph represents this structure through document ranks and directed legal relations.

This structure helps context assembly. When both a primary rule and a lower-level implementing provision are available, the system can include both types of evidence. This is important because a legal answer may require the general rule and the detailed administrative condition.

The final benchmark shows that this modeling is useful, but it does not solve every query. Multi-hop questions and labor-dispute procedure questions still fail when the retriever does not find all required legal bases. Normative hierarchy helps organize retrieved evidence, but it cannot compensate for missing retrieval.

6.5 Citation Grounding and Hallucination Control

Citation hallucination is a central risk in legal QA. The implemented validation layer reduces this risk by checking that answer citations and article coordinates are present in the retrieved context. If the answer cites a legal basis that was not retrieved, the answer fails validation or falls back to safer behavior.

The citation grounding pass rate of 1.000 shows that this layer works well on the final benchmark. The system did not produce unsupported citation surfaces under the deterministic checks.

This result should be interpreted carefully. Citation grounding only checks whether the cited source was retrieved. It does not prove that the retrieved source is the best source or that the answer fully covers all required legal issues. A grounded answer can

still fail if retrieval misses an important citation or if the retrieved context contains a distracting but related provision.

6.6 Practical Applications

The system can be useful as a citation-grounded legal information assistant within the indexed corpus. It can support students, researchers, and HR staff who need to locate relevant statutory passages, retirement-age tables, contract rules, or labor procedure provisions.

The system should be used as a lookup aid, not as a replacement for professional legal advice. Its strongest use case is helping users find and verify legal sources. The final benchmark shows that it is less reliable when the question is informal, when the answer requires a specific table or appendix, or when the query involves labor-dispute procedure. Out-of-corpus QA passes in the benchmark, but this behavior depends on a rule-based guard for known unsupported topic families.

6.7 Limitations

The main limitation is retrieval robustness. The system performs better on direct and structured in-corpus questions than on paraphrased user queries, hard-negative citation cases, multi-hop questions, table and appendix lookup, and labor-dispute procedure questions.

The scope guard improves unsupported-question handling in the benchmark, but it is rule-based. It is limited to known unsupported topic families and should not be treated as a complete detector for every legal topic outside the indexed corpus.

The evaluation is also limited by the corpus and validation method. The corpus contains six indexed documents, so the system cannot answer questions that require external legal instruments. The benchmark is deterministic and does not include human legal expert review. These checks are useful for reproducible software evaluation, but they cannot replace expert assessment of legal interpretation.

Table 6.1: Limitations summary.

Limitation	Thesis Treatment
Scoped corpus	Covers six indexed legal documents and does not cover all Vietnamese law.
Retrieval misses	Report Recall@10, Required Citation Coverage, and failed cases on the final benchmark.
Out-of-corpus behavior	Report refusal tests separately and limit claims to known unsupported topic families.
Deterministic validation	Quality was not reviewed by human legal experts or LLM-as-Judge.
Table and appendix lookup	Identify lookup failures where specific numeric rows are not retrieved.
Legal updates	Newer or external legal sources require corpus refresh and re-evaluation.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

This thesis investigated how hybrid retrieval, legal knowledge graphs, and deterministic validation can support citation-grounded question answering for Vietnamese labor law. Legal QA requires more than fluent summaries. The answer must be linked to the legal source that supports it, and users must be able to verify the document, article, clause, point, or appendix used as the basis.

The project began with employment contract termination, but the final scope was expanded to a broader Vietnamese labor-law assistant. This expansion was necessary because termination questions often depend on related contract rules, retirement rules, minor-worker protections, labor dispute rules, and civil procedure provisions.

The system implements a graph-augmented RAG architecture over six indexed legal documents. It combines hierarchy-aware chunking, deterministic metadata enrichment, hybrid vector-sparse retrieval, a Neo4j legal graph, graph-based retrieval expansion, deterministic intent gating, a rule-based scope guard, and rule-based answer validation. The system is designed to help users locate and verify legal information within the indexed corpus.

The final 100-query benchmark shows both strengths and limitations. The benchmark contains 94 in-corpus queries and 6 out-of-corpus refusal tests. On the in-corpus queries, graph-augmented retrieval reaches Recall@10 of 0.835 and Required Citation Coverage of 0.835. The Forbidden Citation Violation Rate is 0.043, and the in-corpus retrieval pass rate is 0.766. Over the full benchmark, the adjusted end-to-end pass rate is 0.780. The retrieval pass rate is 0.780, the answer pass rate is 0.970, the citation validation pass rate is 1.000, and the quality validation pass rate is 0.970. The average final quality score is 93.9693, with 2 low-information quotes, no unsupported article numbers, and no unretrieved citations. The out-of-corpus QA pass rate is 1.000.

These results show that citation grounding and out-of-corpus refusal are strong in the benchmark. The system is useful as a citation-grounded legal information assistant for questions supported by the indexed corpus. However, the results do not prove perfect legal correctness. The remaining weaknesses are difficult in-corpus retrieval cases, especially paraphrased user questions, hard-negative citation cases, multi-hop questions, table and appendix lookup, and labor-dispute procedure questions.

7.2 Future Work

Future work should improve query rewriting for informal and paraphrased user questions. Many users describe legal problems in everyday language and do not name the relevant article. Better rewriting and legal issue classification could help map these questions to the correct legal provisions.

Table and appendix retrieval also needs further work. Some labor-law answers depend on specific numeric rows, schedules, or forms. Future systems should treat table and appendix content as first-class retrievable evidence rather than ordinary text chunks only.

Hard-negative filtering is another priority. The final benchmark shows that similar but wrong citations can still enter the retrieved context. Future retrieval pipelines should include stronger filtering or reranking for provisions that are lexically similar but legally irrelevant.

Future work should also revisit procedural routing without anchor crowd-out. Labor procedure questions often need several linked legal bases, but forced anchors can distract retrieval if they are added too broadly. A safer version should add procedure context only when it improves the relevant evidence set.

The evaluation should be expanded with an expert-validated benchmark and real user queries. Legal experts can assess whether the answers are legally correct beyond citation containment. Real, anonymized user questions can test informal wording, ambiguity, and practical intent more directly than constructed benchmark items.

Finally, the corpus should be expanded and maintained over time. Vietnamese labor law changes through new laws, decrees, circulars, and administrative guidance. A practical system must support corpus updates, temporal validity checks, and re-evaluation after each major corpus change.

Lessons Learned

The main lesson is that graph expansion is useful, but it is not a universal fix. It can recover related legal provisions, but it still depends on good seed retrieval and careful filtering.

Procedural routing was tested, but it is disabled by default because it introduced retrieval regressions through procedural anchor crowd-out. The final system keeps the more stable scope guard and intent gating configuration.

Relative reference resolution was harder than expected. Cases such as Article 219 show that phrases like “Luật này” can point to external amended laws rather than the local document, so reference extraction needs legal context as well as pattern matching.

References

[1] Ilias Chalkidis et al. "LexGLUE: A Benchmark Dataset for Legal Language Understanding in English". In: *Proceedings of the 2022 Association for Computational Linguistics*. 2021.

[2] Darren Edge et al. "GraphRAG: Unlocking LLM Discovery on Narrative Private Data". In: *arXiv preprint arXiv:2404.16130* (2024).

[3] Shahul Es et al. "Ragas: Automated Evaluation of Retrieval Augmented Generation". In: *arXiv preprint arXiv:2309.15217* (2023).

[4] Neel Guha et al. "LegalBench: A Collaboratively Curated Benchmark Dataset for Measuring Legal Reasoning in Large Language Models". In: *arXiv preprint arXiv:2308.11462* (2023).

[5] keepitreal. *vietnamese-sbert: Sentence-BERT model for Vietnamese*. 2021. url: https://huggingface.co/keepitreal/vietnamese-sbert.

[6] Patrick Lewis et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In: *Advances in Neural Information Processing Systems*. 2020.

[7] National Assembly of Vietnam. *Law No. 80/2015/QH13 on Promulgation of Legislative Documents*. Official Gazette of the Socialist Republic of Vietnam. 2015.

[8] Qdrant Team. *Qdrant Vector Search Engine Documentation*. 2023. url: https://qdrant.tech/documentation/.

[9] Nils Reimers and Iryna Gurevych. "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks". In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. 2019.

[10] Jon Saad-Falcon et al. "ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems". In: *arXiv preprint arXiv:2311.09476* (2023).

[11] Haoxi Zhong et al. "Iterative Contextualization for Legal Question Answering". In: *arXiv preprint arXiv:2009.03846* (2020).

Appendix A

List of Legal Documents

This appendix provides a complete list of the six indexed Vietnamese legal documents comprising the scoped labor-law corpus. The corpus is structured across multiple legislative ranks, spanning primary codes, implementing decrees, and ministerial circulars.

Table A.1: Scoped Vietnamese labor-law corpus documents.

Document ID	Official Title	Document Type	Chunks
45-2019-qh14	Bộ luật Lao động 2019	bo_luat	697
92-2015-qh13-labor-only	Bộ luật Tố tụng dân sự 2015 (subset)	bo_luat	159
nghi-dinh-135-2020-nd-cp	Nghị định 135/2020/NĐ-CP	nghi_dinh	41
nghi-dinh-145-2020-nd-cp	Nghị định 145/2020/NĐ-CP	nghi_dinh	520
thong-tu-09-2020-tt-bldtbxh	Thông tư 09/2020/TT-BLĐTBXH	thong_tu	73
thong-tu-10-2020-tt-bldtbxh	Thông tư 10/2020/TT-BLĐTBXH	thong_tu	66
Total Corpus			**1,556**

Appendix B

Chunk Schema

This appendix documents the object-oriented JSONL schema for preprocessed and enriched evidence units generated by the data processing pipeline. Each chunk encapsulates both hierarchical structural anchors and semantic enrichment attributes.


```
{
  "document_id": "45-2019-qh14",
  "document_title": "Bộ luật số 45/2019/QH14",
  "source_kind": "curated_text",
  "source_path": "corpus/data/curated/45_2019_QH14.txt",
  "chunk_id": "45-2019-qh14-dieu-2-chunk-01",
  "section_id": "45-2019-qh14-dieu-2",
  "article_number": "2",
  "article_title": "Đối tượng áp dụng",
  "heading": "Điều 2. Đối tượng áp dụng",
  "chapter_heading": "Chương I. NHỮNG QUY ĐỊNH CHUNG",
  "section_heading": null,
  "chunk_index": 1,
  "char_count": 121,
  "text": "Điều 2. Đối tượng áp dụng\n\n1. Người lao động, người học nghề, người
  ↪  tập nghề...",
  "clause_ref": "1",
  "point_ref": null,
  "point_refs": [],
  "chunk_type": "clause",
  "parent_chunk_id": null,
  "level": "clause",
  "citation_text": "Bộ luật số 45/2019/QH14, Điều 2 (Đối tượng áp dụng), khoản 1",
  "retrieval_text": "Trong Chương I. NHỮNG QUY ĐỊNH CHUNG, Bộ luật số
  ↪  45/2019/QH14, Điều 2...",
  "page_content": "Trong Chương I. NHỮNG QUY ĐỊNH CHUNG, Bộ luật số 45/2019/QH14,
  ↪  Điều 2...",
  "topic": [],
  "actor": ["nguoi_lao_dong"],
  "issue_type": []
}
```

Appendix C

Graph Schema

This appendix documents the labels, relationship types, properties, and constraints implemented within the Neo4j relational knowledge graph.

C.1 Ontology Labels and Relationships

- **Node Labels**: `Legal_Document`, `Legal_Article`, `Legal_Clause`, `Legal_Point`, `Legal_Appendix`, `Evidence_Chunk`, `Legal_Topic`, `Legal_Actor`, and `Legal_IssueType`.
- **Relationship Types**:
  - *Structural Hierarchy*: `HAS_ARTICLE`, `HAS_CLAUSE`, `HAS_POINT`, `HAS_APPENDIX`.
  - *Source Link*: `HAS_SOURCE_CHUNK` (connecting leaf-level legal anchors to `Evidence_Chunk` nodes).
  - *Cross-Reference*: `REFERENCES` (citations), `DETAILS` (qualifications), and `GUIDED_BY` (administrative delegations).
  - *Taxonomic Metadata*: `MENTIONS_TOPIC`, `APPLIES_TO_ACTOR`, `HAS_ISSUE_TYPE`.
  - *Normative Hierarchy*: `SUPERIOR_TO`, `MUST_COMPLY_WITH`.

C.2 Constraints and Indexes

To ensure structural integrity and quick lookups, unique node constraints are applied to properties:

```
CREATE CONSTRAINT legal_node_node_id IF NOT EXISTS
FOR (n:LegalNode) REQUIRE n.node_id IS UNIQUE;

CREATE CONSTRAINT evidence_chunk_chunk_id_unique IF NOT EXISTS
FOR (n:Evidence_Chunk) REQUIRE n.chunk_id IS UNIQUE;

CREATE INDEX legal_node_node_type IF NOT EXISTS
FOR (n:LegalNode) ON (n.node_type);

CREATE INDEX legal_document_document_id IF NOT EXISTS
FOR (n:Legal_Document) ON (n.document_id);
```

C.3 Diagnostic Cypher Queries

The following Cypher queries are executed by `scripts/inspect_legal_graph.py` to audit database counts:

```
// Count nodes by type
MATCH (n:LegalNode)
RETURN n.node_type AS node_type, count(*) AS count
ORDER BY node_type;

// Count relationships by type
MATCH ()-[r]->()
RETURN type(r) AS edge_type, count(*) AS count
ORDER BY edge_type;
```

Appendix D

Benchmark Samples

This appendix presents representative sample cases extracted from the final 100-query diagnostic evaluation benchmark.

D.1 Case LBR_001: Definition of Employee

- **Query**: Người lao động được định nghĩa như thế nào theo Bộ luật Lao động 2019?
- **Category**: Definition QA
- **Difficulty**: Easy
- **Required Citations**: *Bộ luật Lao động 2019, khoản 1 Điều 3*
- **Gold Answer**: Người lao động là người làm việc cho người sử dụng lao động theo thỏa thuận, được trả lương và chịu sự quản lý, điều hành, giám sát của người sử dụng lao động. Độ tuổi lao động tối thiểu của người lao động là đủ 15 tuổi...

D.2 Case LBR_010: Oral Contract Exception

- **Query**: Trường hợp nào có thể giao kết hợp đồng lao động bằng lời nói?
- **Category**: Exception-based QA
- **Difficulty**: Easy
- **Required Citations**: *Bộ luật Lao động 2019, khoản 2 Điều 14; khoản 2 Điều 18; điểm a khoản 1 Điều 145; khoản 1 Điều 162*
- **Gold Answer**: Hai bên có thể giao kết bằng lời nói đối với hợp đồng có thời hạn dưới 01 tháng. Tuy nhiên, ngoại lệ này không áp dụng đối với người dưới 15 tuổi, người giúp việc gia đình, hoặc nhóm lao động thông qua người ủy quyền.

D.3 Case LBR_PR1: Paraphrased Real-User Query

- **Query**: Cong ty noi hang it viec nen chuyen toi sang viec khac gan 3 thang, khong hoi y kien toi, vay co on khong?
- **Category**: Paraphrased real-user QA

- **Difficulty**: Medium
- **Required Citations**: *Bộ luật Lao động 2019, khoản 1 Điều 29; khoản 2 Điều 29; khoản 3 Điều 29*
- **Purpose**: Tests whether informal wording about temporary transfer to another job is mapped to the correct Labor Code article.

D.4 Case LBR_HN1: Hard Negative

- **Query**: Toi chi muon biet gio nao duoc tinh la lam viec ban dem, khong hoi cach tinh tien luong dem.
- **Category**: Hard-negative citation QA
- **Difficulty**: Medium
- **Required Citations**: *Bộ luật Lao động 2019, Điều 106*
- **Forbidden Citations**: *Bộ luật Lao động 2019, Điều 98; Nghị định 145/2020/NĐ-CP, Điều 56*
- **Purpose**: Tests whether the retriever distinguishes the definition of night work from provisions about night-work wage calculation.

D.5 Case LBR_OOC1: Refusal Query

- **Query**: Nam 2026 luong toi thieu vung I, II, III, IV la bao nhieu?
- **Category**: Out-of-corpus QA
- **Difficulty**: Hard
- **Required Citations**: Empty set
- **Expected Behavior**: The system should refuse or report insufficient indexed context, because the required legal instrument is not present in the indexed corpus.

Appendix E

Evaluation Metrics

This appendix provides the formal mathematical definitions of the evaluation metrics used to measure retrieval effectiveness and response compliance in Chapter 5.

E.1 Information Retrieval Metrics

Let Q represent the full benchmark. The final benchmark is split into in-corpus queries and out-of-corpus queries:

$$Q = Q_{\text{in}} \cup Q_{\text{ooc}}, \qquad |Q| = 100, \quad |Q_{\text{in}}| = 94, \quad |Q_{\text{ooc}}| = 6.$$

Let $G(q)$ represent the set of ground-truth legal citations for a query q, and let $\mathcal{C}_k(q)$ represent the set of citations retrieved within the top-k candidates. Recall@k and Required Citation Coverage are computed only for $q \in Q_{\text{in}}$ where $|G(q)| > 0$. Out-of-corpus queries have empty required citation sets, so citation recall is not defined for them. They are evaluated using refusal or insufficient-context behavior.

Recall@k:

$$\text{Recall}\,@k = \frac{1}{|Q_{\text{in}}|} \sum_{\substack{q \in Q_{\text{in}} \\ |G(q)|>0}} \frac{|G(q) \cap \mathcal{C}_k(q)|}{|G(q)|}$$

Required Citation Coverage:

$$\text{RequiredCitationCoverage}\,@k = \frac{1}{|Q_{\text{in}}|} \sum_{\substack{q \in Q_{\text{in}} \\ |G(q)|>0}} \frac{|G(q) \cap \mathcal{C}_k(q)|}{|G(q)|}$$

Forbidden Citation Violation Rate: Let $F(q)$ define the set of legally obsolete or distracting citations for query q:

$$\text{ForbiddenViolationRate} = \frac{1}{|Q_{\text{in}}|} \sum_{q \in Q_{\text{in}}} \mathbb{I}\left[F(q) \cap \mathcal{C}_k(q) \neq \emptyset\right]$$

where $\mathbb{I}\left[\cdot\right]$ is the boolean indicator function.

E.2 End-to-End System Pass Logic

For an in-corpus query, the pass state combines retrieval, answer, citation, and quality checks:

$$\text{Pass}_{\text{in}}(q) = \text{Pass}_{\text{retrieval}}(q) \wedge \text{Pass}_{\text{answer}}(q) \wedge \text{Pass}_{\text{citation}}(q) \wedge \text{Pass}_{\text{quality}}(q)$$

where each term denotes the boolean outcome of the retrieval layer, the answer presence gate, the citation containment check, and the token density quality check.

For an out-of-corpus query, RefusalPass(q) is true only when the system refuses or reports insufficient indexed context. The adjusted end-to-end score is:

$$\text{AdjustedE2E} = \frac{\sum_{q \in Q_{\text{in}}} \mathbb{I}[\text{Pass}_{\text{in}}(q)] + \sum_{q \in Q_{\text{ooc}}} \mathbb{I}[\text{RefusalPass}(q)]}{|Q|}$$

For the final benchmark:

$$\text{AdjustedE2E} = \frac{78}{100} = 0.780.$$

Appendix F

Reproducibility Commands

This appendix documents the shell commands required to execute evaluations, rebuild indices, or regenerate database structures in PowerShell.

F.1 Final Candidate Configuration

The final candidate uses scope guard and intent gating. The following routing setting is used for the final benchmark:

```
enable_procedural_routing: false
```

F.2 Rerun Retrieval Ablation

```
.venv\Scripts\python.exe -X utf8 scripts/ablation_retrieval_100.py `
  --benchmark-path artifacts/evaluation/golden_benchmark_100_extended.jsonl `
  --output-dir artifacts/evaluation `
  --output-prefix ablation_retrieval_100_final_candidate `
  --top-k 10 `
  --prefetch-limit 24 `
  --device cpu `
  --embedding-provider sentence_transformers
```

F.3 Rerun End-to-End Evaluation

```
.venv\Scripts\python.exe -X utf8 scripts/evaluate_end_to_end_rag.py `
  --benchmark-path artifacts/evaluation/golden_benchmark_100_extended.jsonl `
  --output-dir artifacts/evaluation `
  --output-prefix end_to_end_100_final_candidate `
  --top-k 10 `
  --prefetch-limit 24 `
  --max-answer-contexts 8 `
  --provider extractive `
  --device cpu `
  --embedding-provider sentence_transformers
```

F.4 Adjusted Split Metrics

The final score is the adjusted end-to-end pass rate, where in-corpus queries use the normal retrieval, answer, citation, and quality checks, and out-of-corpus queries use refusal

or insufficient-context validation:

$$\text{AdjustedE2E} = 0.780.$$

The adjusted split metrics are recomputed from the final candidate results JSON using the rules in Appendix E:

```
$evalDir = "artifacts/evaluation"
$results = "$evalDir/end_to_end_100_final_candidate_results.json"
$outJson = "$evalDir/benchmark_100_final_candidate_split_metrics.json"
$outCsv = "$evalDir/benchmark_100_final_candidate_failed_cases.csv"

$splitArgs = @(
  "--results-path", $results
  "--output-json", $outJson
  "--failed-cases-csv", $outCsv
)
.venv\Scripts\python.exe -X utf8 scripts/compute_100_split_metrics.py @splitArgs
```

The command writes the JSON summary and failed-case CSV under `artifacts/evaluation/`.

F.5 Optional: Regenerate Qdrant Vector Index

```
.venv\Scripts\python.exe scripts/build_index.py `
  --chunk-file artifacts/chunks/legal_chunks_enriched.jsonl `
  --artifacts-dir artifacts `
  --dense-model keepitreal/vietnamese-sbert `
  --collection-name vietnamese_labor_law_chunks `
  --device cpu
```

F.6 Optional: Rebuild Neo4j Legal Graph

```
.venv\Scripts\python.exe scripts/build_legal_graph.py `
  --chunks-path artifacts/chunks/legal_chunks_enriched.jsonl `
  --reference-edges-path artifacts/graph/reference_edges.jsonl `
  --summary-path artifacts/graph/legal_graph_build_summary.json `
  --with-concepts `
  --with-references `
  --with-normative-hierarchy `
  --reset
```

Appendix G

Example Retrieved Contexts and Answers

This appendix provides a concrete runtime trace illustrating a retrieved context block and the corresponding structured output payload generated by the extractive fallback.

G.1 Example Input Query

"Người lao động được định nghĩa như thế nào theo Bộ luật Lao động 2019?"

G.2 Retrieved Evidence Chunk

```
{
  "chunk_id": "45-2019-qh14-dieu-3-chunk-02",
  "citation_text": "Bộ luật số 45/2019/QH14, Điều 3 (Giải thích từ ngữ), khoản 1",
  "text": "1. Người lao động là người làm việc cho người sử dụng lao động \n  theo
  ↪  thỏa thuận, được trả lương và chịu sự quản lý, điều hành, giám sát \n  của
  ↪  người sử dụng lao động. Độ tuổi lao động tối thiểu của người \n  lao động là
  ↪  đủ 15 tuổi, trừ trường hợp quy định tại Mục 1 Chương XI..."
}
```

G.3 Validated Generated Output

```
{
  "answer": "Căn cứ Bộ luật số 45/2019/QH14, Điều 3, khoản 1, người lao động \n
  ↪  được định nghĩa là người làm việc cho người sử dụng lao động theo thỏa
  ↪  thuận, \n  được trả lương và chịu sự quản lý, điều hành, giám sát của người
  ↪  sử \n  dụng lao động. Độ tuổi lao động tối thiểu là đủ 15 tuổi.",
  "legal_basis": ["Bộ luật số 45/2019/QH14, Điều 3 (Giải thích từ ngữ), khoản 1"],
  "evidence_quotes": [
    {
      "citation": "Bộ luật số 45/2019/QH14, Điều 3, khoản 1",
      "quote": "Người lao động là người làm việc cho người sử dụng lao động..."
    }
  ],
  "insufficient_context": false
}
```